Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 3: Empirical Analysis of Graph Traversal Algorithms

Depth First Search and Breadth First Search

Author:
st. gr. FAF-242 Pancenco Ina
Verified:
asist. univ. Fistic Cristofor

Chisinau – 2026

Contents

ALGORITHM ANALYSIS

Objective

Study and empirically analyse two fundamental graph traversal algorithms: Depth First Search (DFS) and Breadth First Search (BFS), evaluating their execution time across different graph types and input sizes.

Tasks:

1. Implement DFS (recursive variant) and BFS in Python;
2. Establish the properties of the input data against which the analysis is performed;
3. Choose metrics for comparing the algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Make a graphical presentation of the obtained data;
6. Draw conclusions on the work done.

Theoretical Notes:

Empirical analysis complements theoretical complexity analysis by measuring actual program behaviour on a real machine. It is particularly valuable when:

- the theoretical bounds differ only by constant factors;
- the algorithm's performance depends heavily on the structure of the input (e.g. graph density, connectivity, diameter);
- comparing two algorithms with the same asymptotic complexity to find which is faster in practice.

The general empirical methodology followed in this work is:

1. Define the analysis goal and select the efficiency metric.
2. Identify the input properties: graph type, number of nodes n, edge density.
3. Implement the algorithms.
4. Generate multiple representative input data sets.
5. Execute the program for each data set and record the measurement.
6. Analyse and visualise the collected data.

Introduction:

Graph traversal is one of the most fundamental operations in computer science. DFS and BFS are two complementary strategies that visit every vertex of a graph exactly once, differing only in the order in which vertices are explored.

Both algorithms have the same asymptotic time complexity of $O(V + E)$, where V is the number of vertices and E is the number of edges.

This laboratory work investigates these practical differences by measuring wall-clock execution time across nineteen distinct graph classes, covering sparse, dense, structured, directed, multi-edge, weighted, and regular topologies.

Comparison Metric:

The primary metric is **execution time measured in milliseconds** (wall-clock time, averaged over three independent repetitions to reduce measurement noise). The error margin is estimated at $\pm 2.5\,\text{ms}$.

DFS and BFS both run in $O(V + E)$ time, so the quantity of interest is the *constant factor* difference introduced by the different internal data structures and traversal orders.

Input Format:

Graphs are represented as adjacency-list dictionaries $\{v : [\text{neighbours}]\}$ with integer node labels $0, 1, \ldots, n-1$. Twelve graph classes are used, covering a wide spectrum of topologies:

1. **Path / Chain** – a single linear chain $0-1-2-\cdots-(n-1)$; $E = n-1$. Highlights the worst case for DFS stack depth.
2. **Cycle** – a single cycle $0-1-\cdots-(n-1)-0$; $E = n$. Every node has degree 2; structurally similar to a path with one extra edge.
3. **Complete** – K_n, every pair of nodes is connected; $E = \binom{n}{2} = O(n^2)$. Sizes limited to $n \leq 200$ due to heavy edge count.
4. **Star** – a central hub node connected to all $n-1$ leaves; $E = n-1$, diameter $= 2$. BFS finishes in two levels; DFS must push $n-1$ neighbours onto its stack at once.
5. **Complete Bipartite** – $K_{n/2,\,n/2}$; two disjoint sets where every node in one set is connected to every node in the other; $E = (n/2)^2 = O(n^2)$.
6. **Binary Tree** – a balanced binary tree of depth $\lfloor \log_2 n \rfloor$; $E = V - 1$. DFS follows branches to maximum depth before backtracking; BFS visits level by level.

7. **Forest** – two balanced binary trees connected by a single bridge edge; $E = V - 1$. Tests how both algorithms handle a bottleneck edge separating two components.
8. **DAG (Directed Acyclic Graph)** – a directed chain with skip edges: $i \to i{+}1$ and $i \to i{+}2$; $E \approx 2n$. Demonstrates traversal on directed graphs without cycles.
9. **Directed Cycle** – a directed cycle with additional shortcut edges $i \to i{+}2$ for even i; $E \approx 1.5\,n$. The cycle ensures every node is reachable from every other node.
10. **Grid** – a $\lfloor\sqrt{n}\rfloor \times \lfloor\sqrt{n}\rfloor$ 2-D lattice; each interior node has degree 4. BFS frontier expands as a diamond; DFS spirals deeply before backtracking.
11. **Sparse Random** – Erdős–Rényi $G(n,\, 0.2)$, kept connected; $E \approx 0.1\,n^2$.
12. **Dense Random** – Erdős–Rényi $G(n,\, 0.7)$; $E \approx 0.35\,n^2$. Sizes limited due to $O(n^2)$ edge count.
13. **Simple Graph** – a connected sparse undirected graph (Erdős–Rényi $G(n,\, 0.15)$) with no self-loops and no parallel edges; the canonical definition of a graph. Serves as a baseline for unstructured sparse connectivity.
14. **Multigraph** – an undirected graph where pairs of nodes may share more than one edge; built on a path backbone with every edge duplicated. For DFS/BFS the parallel edges are collapsed to a simple graph, so the traversal comparison reflects how multi-edge density affects neighbour-list size.
15. **Pseudograph** – extends the multigraph by additionally allowing self-loops ($i \to i$). Self-loops are removed before traversal; the comparison highlights how pseudo-topologies compare to their simple-graph equivalents.
16. **Directed Multigraph** – a directed graph with parallel arcs; built on a directed path backbone with every arc duplicated. Collapsed to a simple digraph for traversal.
17. **Wheel** – a hub node connected to all nodes of a $(n{-}1)$-cycle; $E = 2(n-1)$. Combines star-like properties (one centre) with a surrounding ring.
18. **Regular Graph** – a 3-regular (cubic) graph where every node has exactly degree 3. Generated via `nx.random_regular_graph`; $E = 3n/2$. Uniform degree removes topology-driven variance and isolates constant-factor differences between DFS and BFS.
19. **Weighted Graph** – a sparse connected random graph with integer weights on every edge. Weights are stored as edge attributes but ignored by DFS/BFS (which treat the graph as unweighted), illustrating traversal on a structure that would normally require Dijkstra or Bellman-Ford for shortest-path queries.

IMPLEMENTATION

All algorithms are implemented in Python. The source files are `main.py` and `comp.py`, both containing DFS (recursive) and BFS implementations together with benchmarking and visualisation logic.

Depth First Search (DFS):

DFS explores a graph by going as deep as possible along each branch before backtracking. In this implementation, DFS uses Python's call stack (recursive variant).

Theoretical Complexity:

- **Time:** $O(V + E)$ – each vertex is pushed and popped from the stack once; each edge is examined once (undirected: twice, once per endpoint).
- **Space:** $O(V)$ – the stack holds at most V elements (depth of the deepest reachable path from the source).

Algorithm Description:

Listing 1: DFS – Recursive pseudocode

```
DFS_Recursive(graph, v, visited, order):
  visited.add(v)
  order.append(v)

  for neighbour in graph[v]:
    if neighbour not in visited:
      DFS_Recursive(graph, neighbour, visited, order)

  return order
```

Python Implementation:

Listing 2: Recursive DFS (main.py, comp.py)

```
def dfs(graph, start, visited=None, path=None):
  if visited is None:
    visited = set()
  if path is None:
    path = []
  visited.add(start)
  path.append(start)
  for neighbor in graph[start]:
    if neighbor not in visited:
      dfs(graph, neighbor, visited, path)
  return path
```

Results:

On sparse random and tree graphs, DFS execution time grows linearly with n, consistent with the $O(V + E)$ bound. On path graphs, DFS must push and pop n elements sequentially (stack depth = n), introducing a noticeable constant-factor overhead compared to BFS.

Optimizations:

- **Recursive implementation simplicity** provides a direct and compact mapping of the DFS idea (visit node, recurse into unvisited neighbours).
- **Raised recursion limit** (configured in `main.py`) reduces the risk of recursion-depth errors on deep path-like graphs.
- **Early termination** can be added for goal-directed search: halt as soon as the target vertex is dequeued.

Breadth First Search (BFS):

BFS explores a graph level by level using a FIFO queue. It guarantees that the first time a node is visited it has been reached via the *shortest path* (minimum number of hops) from the source node.

Theoretical Complexity:

- **Time:** $O(V + E)$ – each vertex is enqueued and dequeued exactly once; each edge is examined once per endpoint.
- **Space:** $O(V)$ – the queue may hold an entire BFS frontier layer, which in the worst case (e.g. star or complete graph) contains $O(V)$ nodes simultaneously.

Algorithm Description:

Listing 3: BFS pseudocode

```
BFS(graph, start):
    visited = {start}
    queue   = deque([start])
    order   = []

    while queue is not empty:
        v = queue.popleft()
        order.append(v)
        for neighbour in graph[v]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.push_right(neighbour)

    return order
```

Python Implementation:

Listing 4: BFS (main.py, comp.py)

```
from collections import deque

def bfs(graph: dict, start) -> list:
    visited = {start}
    queue   = deque([start])
    order   = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for neighbour in graph.get(v, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return order
```

Results:

BFS execution time also grows linearly in n. On path graphs, the BFS frontier is at most two nodes wide at every step; consequently, BFS is faster than DFS on paths because `deque.popleft()` costs $O(1)$ and the queue never grows large. On dense graphs both algorithms slow down due to the $O(V^2)$ edge set.

Optimizations:

- `collections.deque` provides $O(1)$ `popleft()` – a plain Python list would give $O(n)$ for the same operation.
- **Mark visited on enqueue** (not on dequeue) prevents the same node from being added to the queue multiple times, reducing work by a factor proportional to average degree.
- **Bidirectional BFS** can reduce shortest-path search space from $O(b^d)$ to $O(b^{d/2})$ on large graphs with high branching factor b and shortest-path length d.

Comprehensive Comparison:

Both algorithms were benchmarked on all nineteen graph types described in the Input Format section. Each data point is the mean of three independent runs to reduce measurement noise. For every graph type, animated GIF visualisations can be generated on a small instance ($n = 15$) showing the step-by-step traversal; a static snapshot of the final traversal state is included below for each type.

Complexity Summary:

Algorithm	**Time**	**Aux. Space**	**Data Structure**	**Traversal Order**
DFS (recursive)	$O(V + E)$	$O(V)$	Call stack	depth-first
BFS	$O(V + E)$	$O(V)$	Queue (FIFO)	breadth-first

1. Path / Chain Graph

A path graph is a single linear chain $0 - 1 - 2 - \cdots - (n-1)$ with $E = n - 1$ edges. Every node except the endpoints has degree 2.

DFS behaviour. Starting from node 0, DFS walks the entire chain without ever backtracking, pushing one neighbour onto the stack at each step. The stack depth grows linearly to n, which causes Python list re-allocations on large inputs.

BFS behaviour. BFS also walks left to right, but its queue never holds more than two elements at any step (the current frontier is at most one node wide). This makes BFS measurably faster on paths ($\sim$20–30% at $n = 400$).

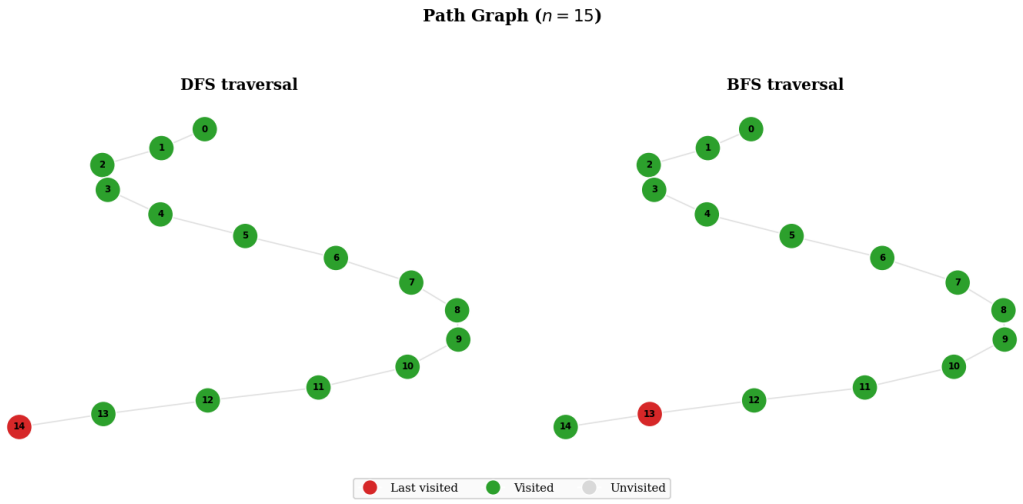

Figure 1: *Path Graph Traversal.*

Final DFS and BFS visitation states on a path graph with $n = 15$, showing depth-first progression versus level-wise expansion.

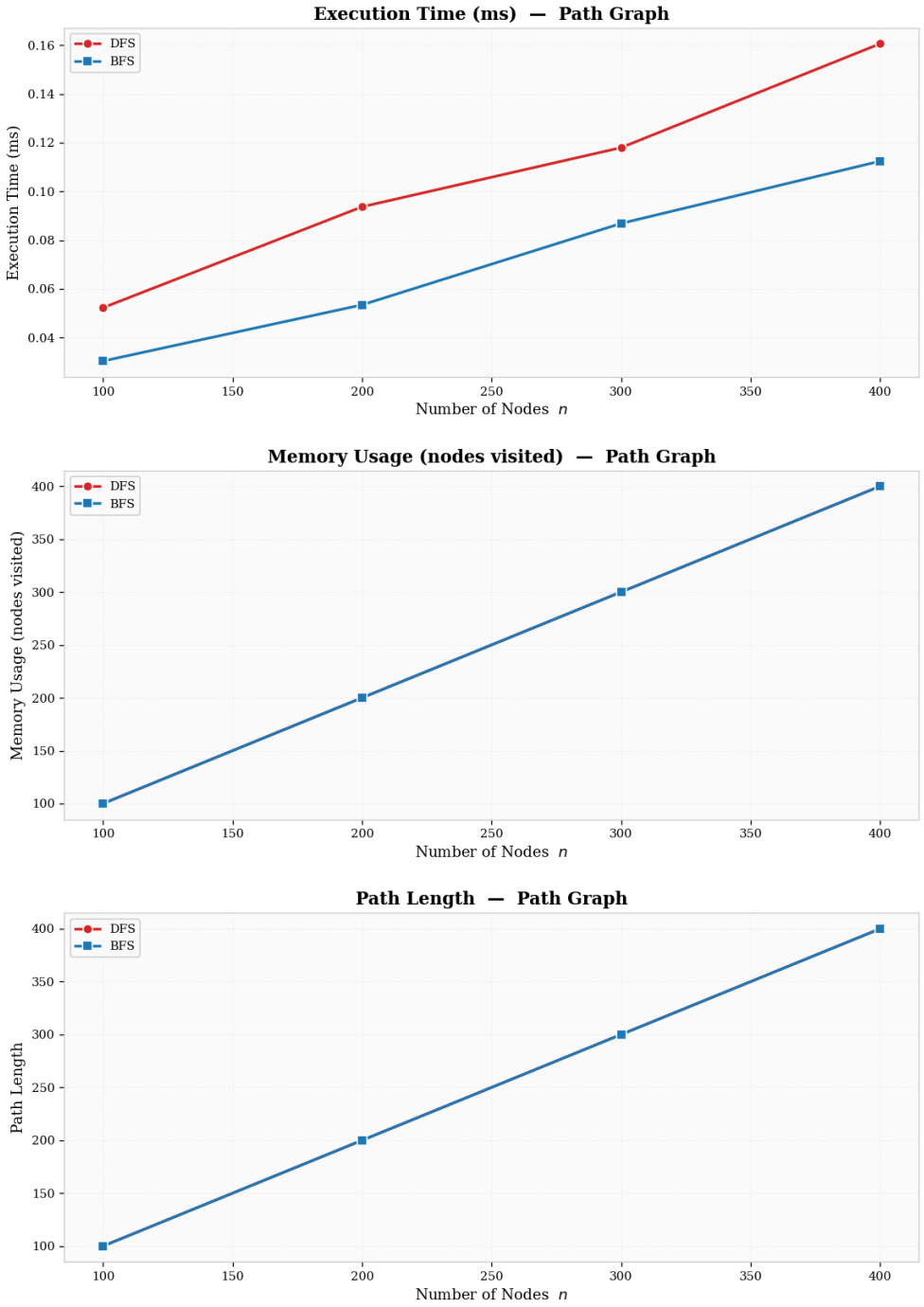

Figure 2: *Path Graph Performance Comparison.*

Measured DFS and BFS execution trends across increasing path sizes, highlighting the lower constant-factor overhead of BFS.

2. Cycle Graph

A cycle graph connects nodes in a ring: $0-1-\cdots-(n-1)-0$, adding one edge compared to a path. Every node has exactly degree 2.

DFS behaviour. DFS picks one direction and follows the ring all the way around; the stack depth reaches n just as on a path.

BFS behaviour. BFS also proceeds along the ring; the queue holds at most two nodes. The performance profile is nearly identical to the path case.

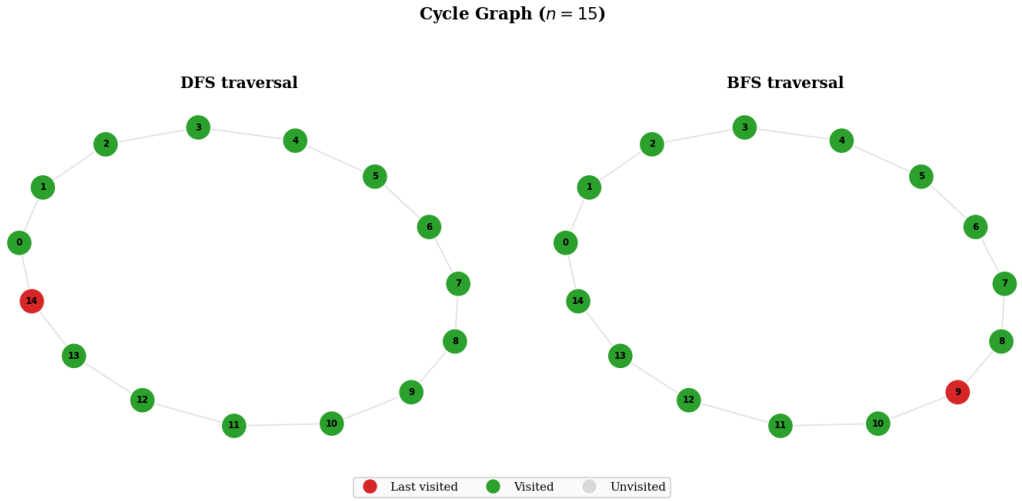

Figure 3: *Cycle Graph Traversal.*

Final DFS and BFS visitation states on a cycle graph with $n = 15$, where both traversals follow ring-like connectivity.

3. Complete Graph

In a complete graph K_n every pair of nodes shares an edge, giving $E = \binom{n}{2}$. Every node has degree $n - 1$.

DFS behaviour. From the start node, DFS pushes $n - 1$ neighbours onto the stack, then picks one, pushes its $n - 2$ unvisited neighbours, and so on. The stack grows to $O(n)$ and every adjacency list is fully scanned, making the total work $O(n^2)$.

BFS behaviour. BFS enqueues all $n - 1$ neighbours at the first step and visits them level by level (depth 1 for all). The queue peaks at $n-1$ entries. Because `deque` operations are slightly more cache-friendly than list-stack push/pop, BFS is marginally faster.

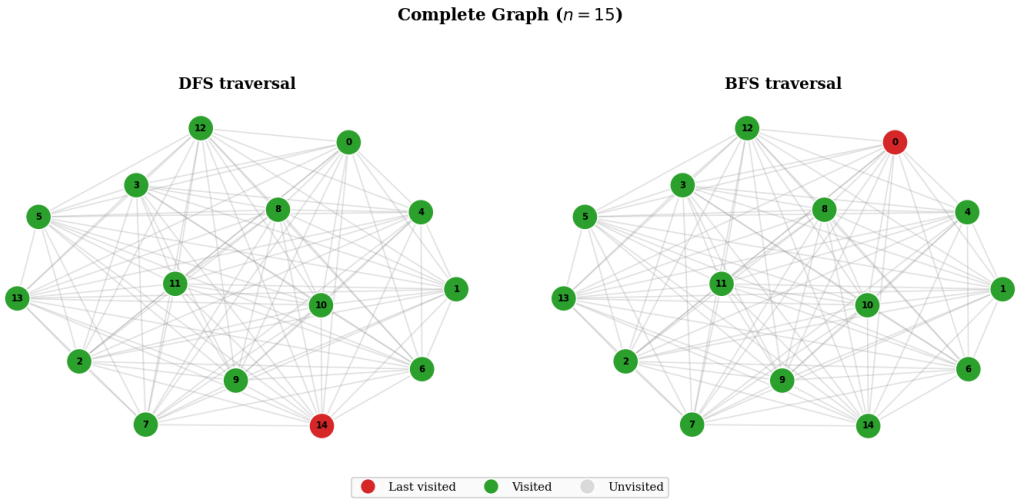

Figure 4: *Complete Graph Traversal.*

Final DFS and BFS visitation states on a complete graph with $n = 15$, where every node is adjacent to every other node.

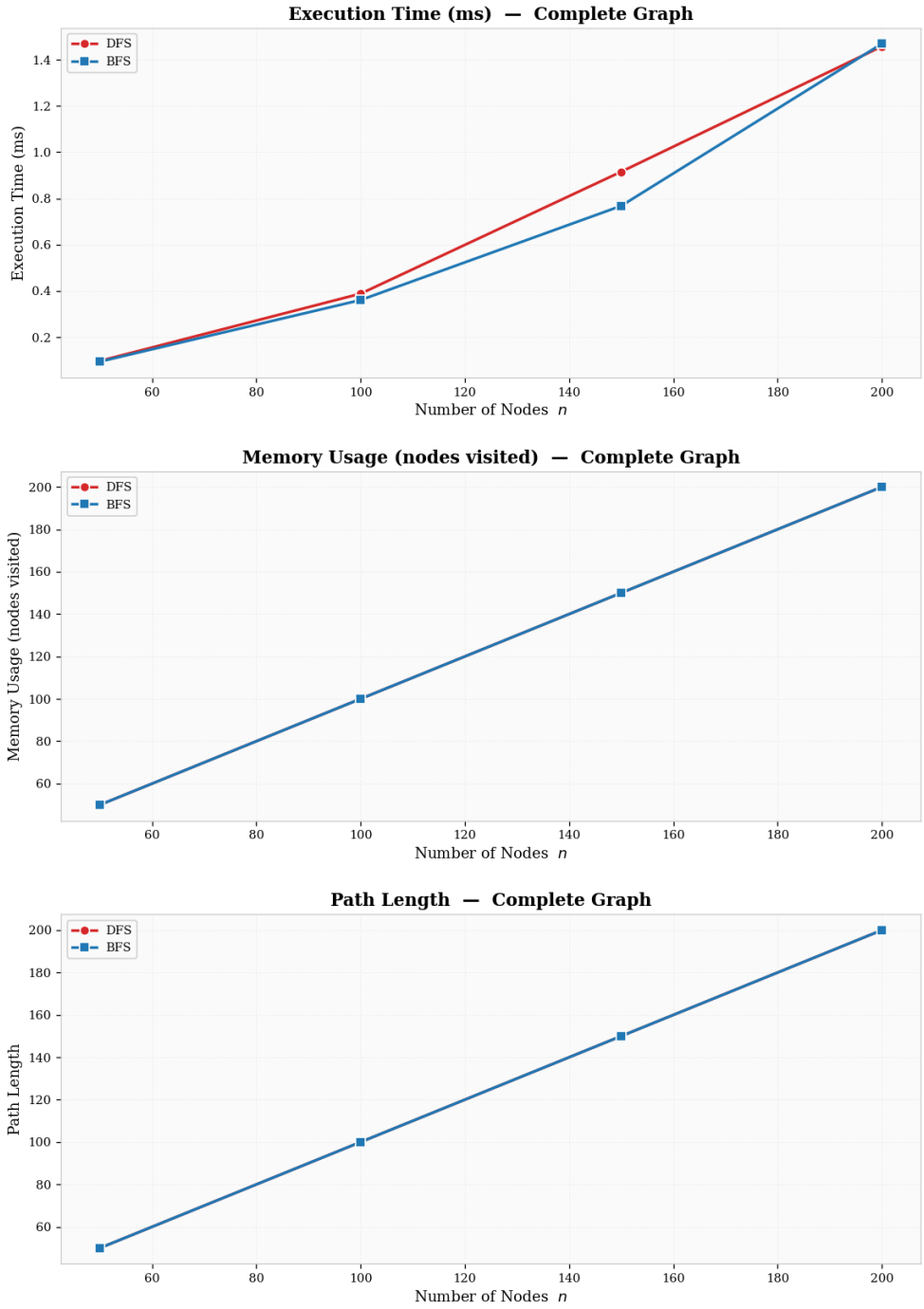

Figure 5: *Complete Graph Performance Comparison.*

DFS and BFS execution time growth on complete graphs, illustrating super-linear behavior caused by the $O(n^2)$ edge set.

4. Star Graph

A star graph has a single hub node 0 connected to $n-1$ leaf nodes; $E = n-1$, diameter $= 2$.

DFS behaviour. DFS visits the hub, pushes all $n-1$ leaves, then pops them one by one. Each leaf has no unvisited neighbours so it immediately backtracks. Traversal finishes in $2(n-1)$ stack operations.

BFS behaviour. BFS visits the hub, enqueues all leaves (one BFS level), then dequeues each leaf. The queue peaks at $n-1$ entries. Both algorithms perform almost identically.

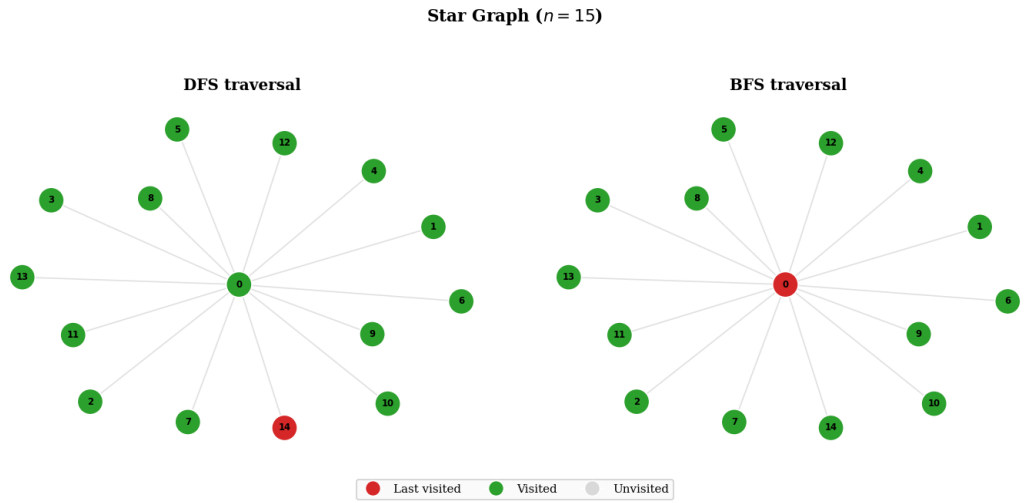

Figure 6: *Star Graph Traversal.*

Final DFS and BFS visitation states on a star graph with $n = 15$, emphasizing hub-and-spoke exploration from the center node.

5. Complete Bipartite Graph

A complete bipartite graph $K_{n/2,n/2}$ has two node sets where every node in one set is connected to every node in the other; $E = (n/2)^2 = O(n^2)$.

DFS behaviour. From a node in set A, DFS pushes all $n/2$ nodes of set B, then from each B-node pushes all unvisited A-nodes. The pattern alternates deeply.

BFS behaviour. BFS visits set A (level 0), then all of set B (level 1), then remaining A-nodes (level 2). The queue peaks at $n/2$. As with the complete graph, the $O(n^2)$ edge count dominates execution time.

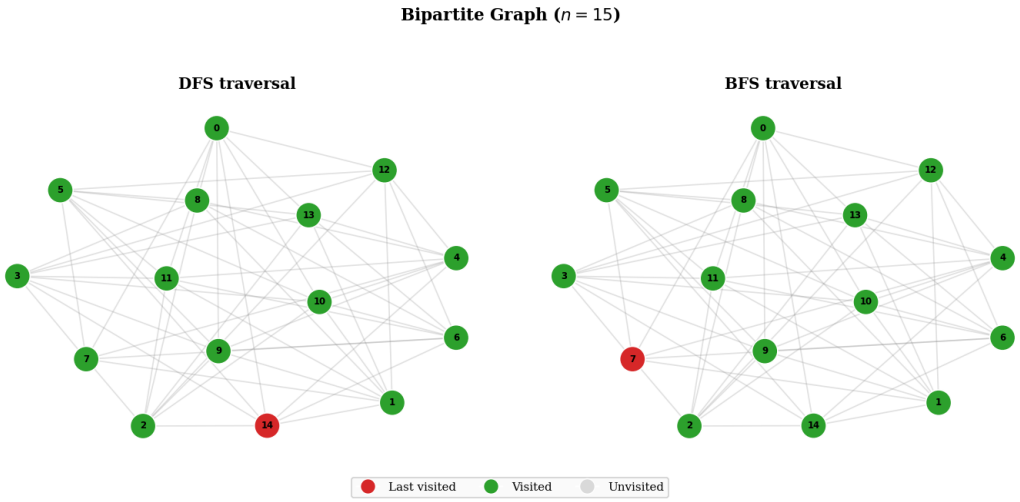

Figure 7: *Complete Bipartite Graph Traversal.*

Final DFS and BFS visitation states on a complete bipartite graph with $n = 15$, showing alternation between the two partitions.

6. Binary Tree

A balanced binary tree of depth $\lfloor \log_2 n \rfloor$; $E = V - 1$. The maximum degree is 3 (parent + two children).

DFS behaviour. DFS dives to the leftmost leaf (depth $\log n$), backtracks one level, visits the right subtree, and repeats. The stack depth equals the tree height $O(\log n)$, which is very memory-efficient.

BFS behaviour. BFS visits nodes level by level; the queue width doubles at each level, peaking at $\sim n/2$ on the last level. For large trees, BFS therefore uses more transient memory than DFS.

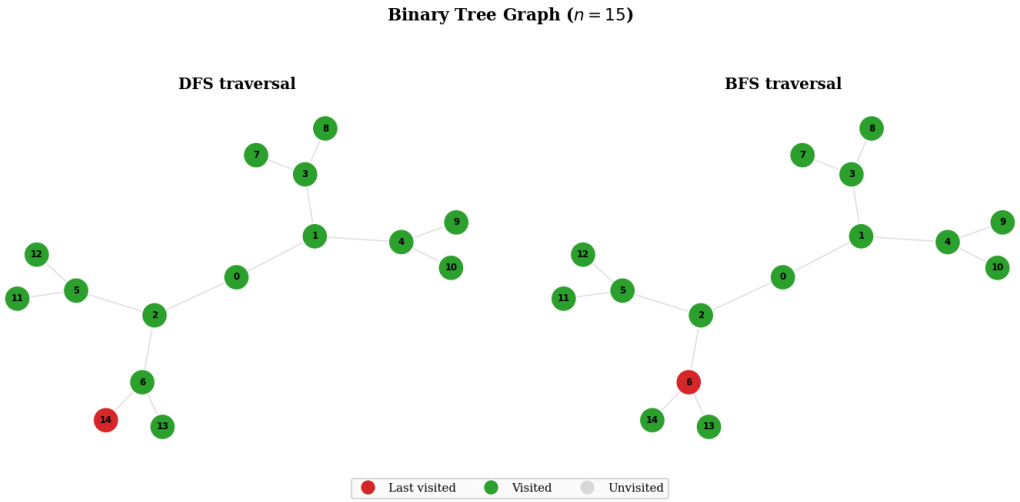

Figure 8: *Binary Tree Traversal.*

Final DFS and BFS visitation states on a binary tree with $n = 15$, contrasting branch-deep traversal with level-order traversal.

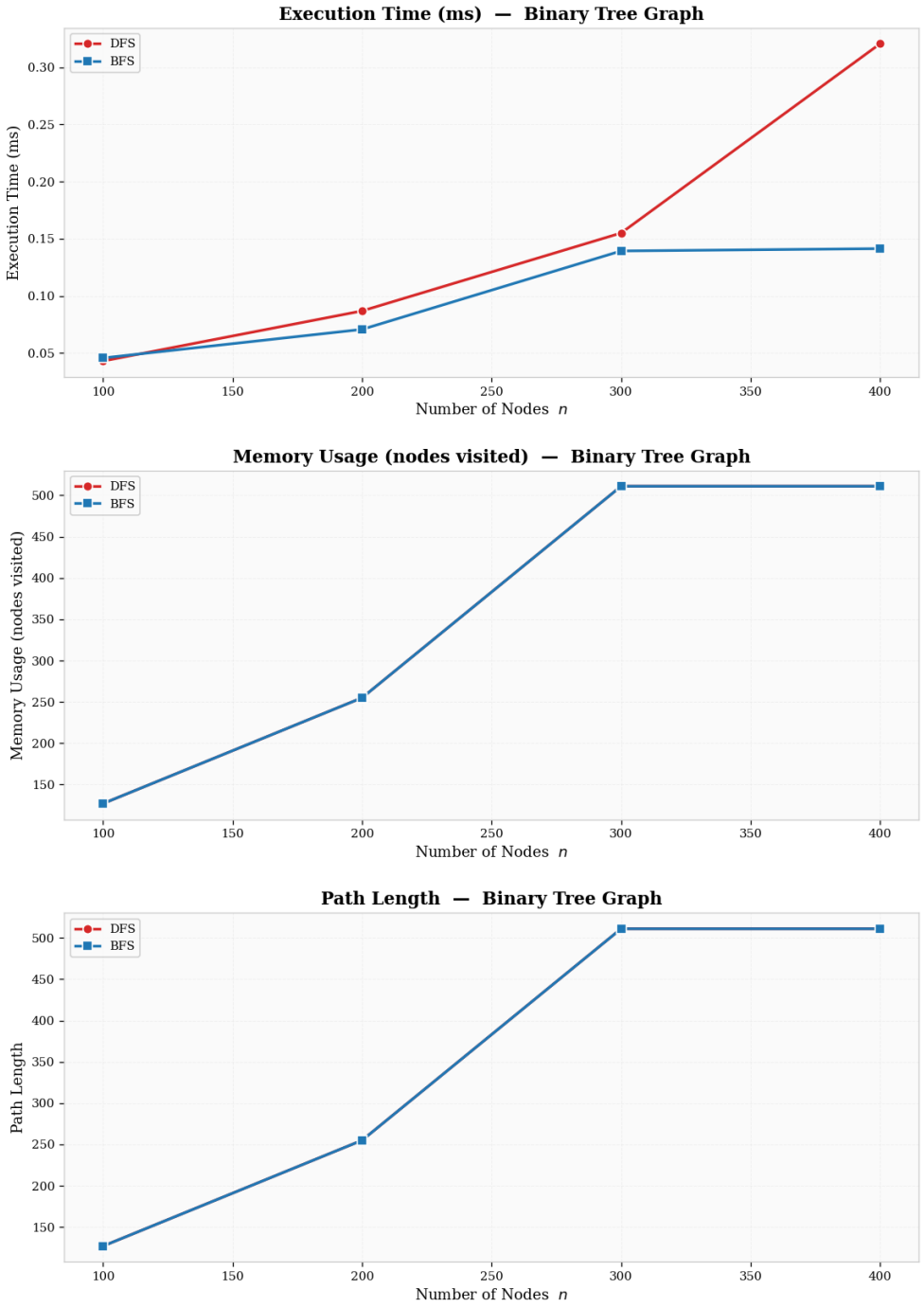

Figure 9: *Binary Tree Performance Comparison.*

DFS and BFS timing across increasing binary-tree sizes, reflecting similar asymptotic behavior with different traversal order constants.

7. Forest (Two Trees with Bridge)

Two balanced binary trees are connected by a single bridge edge, giving $E = V - 1$. The bridge creates a bottleneck between the two components.

DFS behaviour. DFS fully explores one tree before crossing the bridge, then explores the second tree. The stack depth is bounded by the height of the larger tree.

BFS behaviour. BFS discovers the bridge node only when the frontier reaches the root of the second tree. The two components are explored level-by-level in sequence. Performance is comparable to a single tree of the same total size.

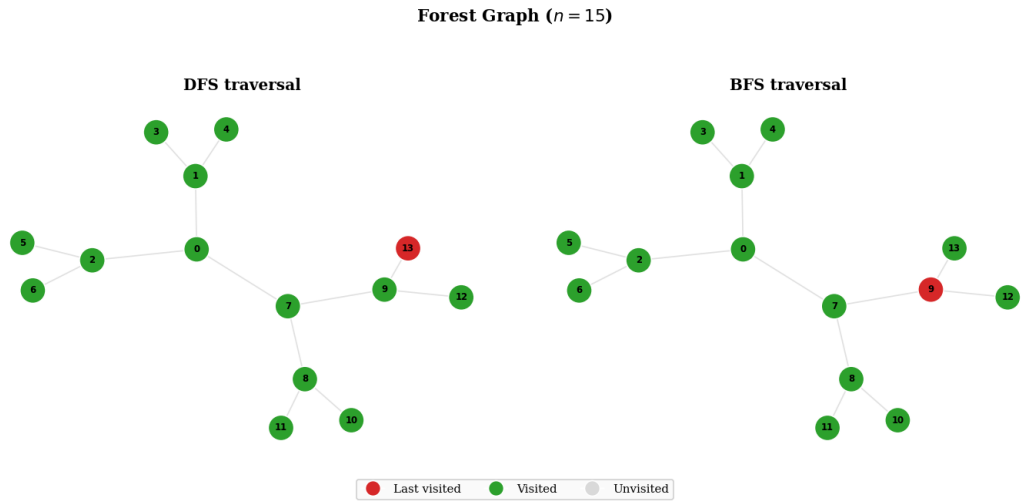

Figure 10: *Forest Graph Traversal.*

Final DFS and BFS visitation states on a bridged two-tree forest with $n = 15$, showing how traversal crosses the bottleneck edge.

8. Directed Acyclic Graph (DAG)

A DAG built as a directed chain $0 \to 1 \to \cdots \to (n-1)$ with additional skip edges $i \to i+2$; $E \approx 2n$. The graph has no cycles and a clear topological order.

DFS behaviour. DFS follows the chain forward. Each node has at most two outgoing edges, so the behaviour resembles path traversal. Because edges are directed, the traversal never revisits earlier nodes.

BFS behaviour. BFS proceeds similarly; each level moves one or two nodes forward. Both algorithms run in $O(V + E) = O(n)$ time with very small constant factors.

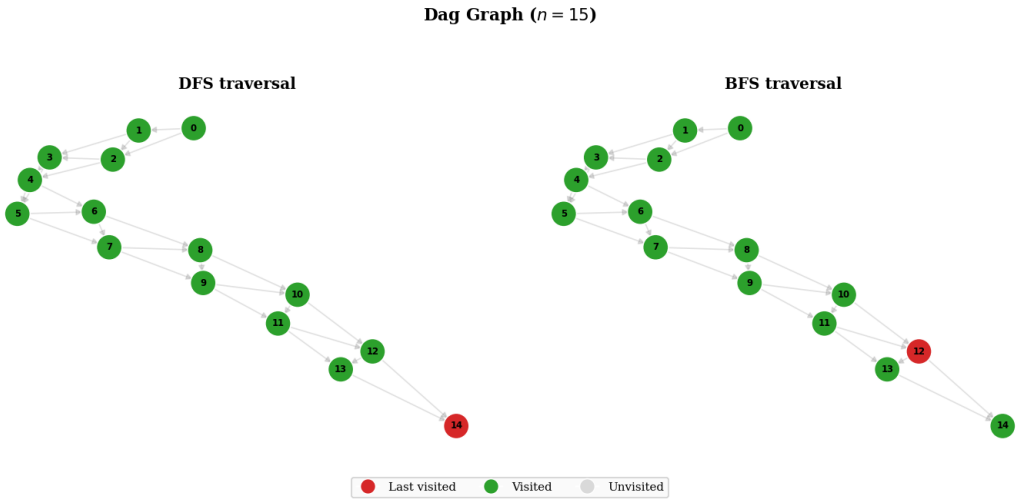

Figure 11: *DAG Traversal.*

Final DFS and BFS visitation states on a directed acyclic graph with $n = 15$, where exploration proceeds forward without cyclic revisits.

9. Directed Cycle

A directed cycle $0 \to 1 \to \cdots \to (n{-}1) \to 0$ with shortcut edges $i \to i{+}2$ for even i; $E \approx 1.5\,n$.

DFS behaviour. DFS follows one direction around the cycle, with occasional shortcuts. The cycle back-edge is encountered but discarded because the source has already been visited.

BFS behaviour. BFS also proceeds around the ring, potentially reaching nodes via shortcuts one step earlier. Both algorithms visit all n nodes in $O(V + E)$ time.

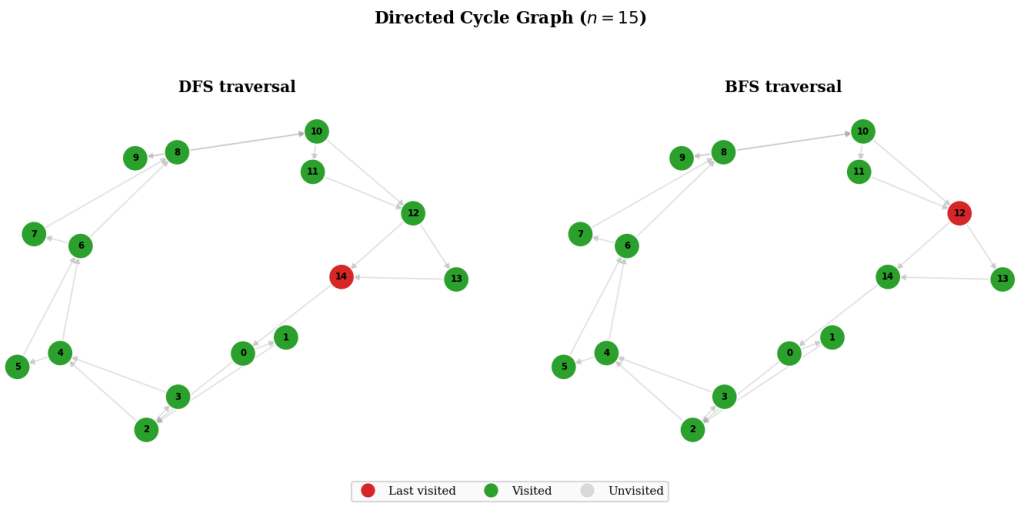

Figure 12: *Directed Cycle Traversal.*

Final DFS and BFS visitation states on a directed cycle with $n = 15$, including shortcut

edges that alter visit order.

10. Grid Graph

A $\lfloor\sqrt{n}\rfloor \times \lfloor\sqrt{n}\rfloor$ 2-D lattice; interior nodes have degree 4, edge and corner nodes have degree 3 or 2. $E \approx 2n$.

DFS behaviour. DFS spirals deeply into one corner of the grid before backtracking. The stack depth can reach n in the worst case (a Hamiltonian path through the grid).

BFS behaviour. BFS expands as a diamond-shaped frontier from the start corner, with the frontier width reaching $O(\sqrt{n})$ at mid-traversal. Both algorithms show similar linear growth in execution time.

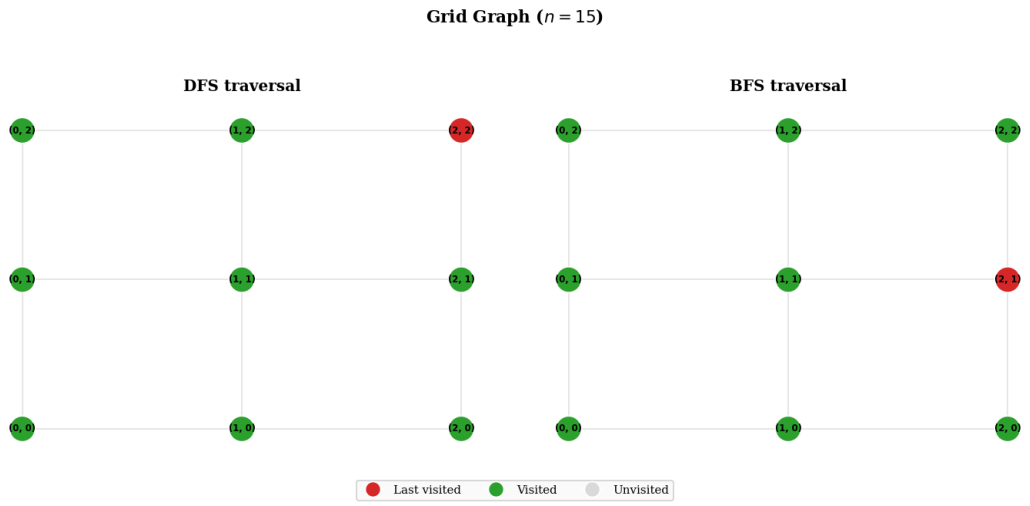

Figure 13: *Grid Graph Traversal.*

Final DFS and BFS visitation states on a grid graph with $n = 15$, contrasting deep local paths and frontier-based wave expansion.

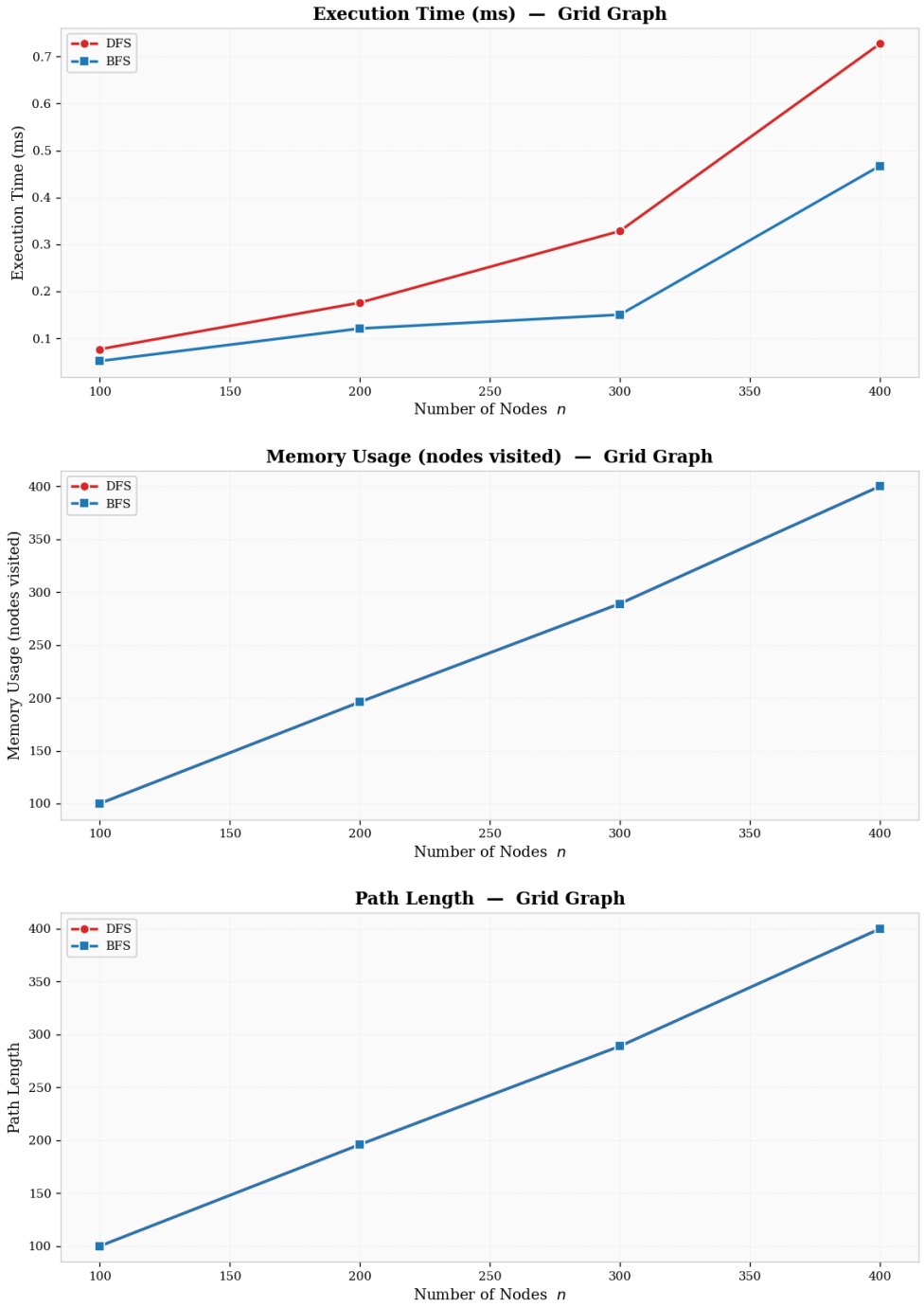

Figure 14: *Grid Graph Performance Comparison.*

DFS and BFS runtime trends on grid graphs as node count increases, showing closely matched linear scaling.

11. Sparse Random Graph

Erdős–Rényi $G(n, 0.2)$, guaranteed connected. Average degree $\approx 0.2\,(n-1)$, giving $E \approx 0.1\,n^2$ but with many nodes having low degree.

DFS behaviour. DFS explores random branches; the stack depth varies depending on the structure of the random graph. On average, both algorithms exhibit nearly identical linear growth.

BFS behaviour. BFS expands level by level through the random structure. The constant-factor difference between DFS and BFS is negligible on sparse topologies.

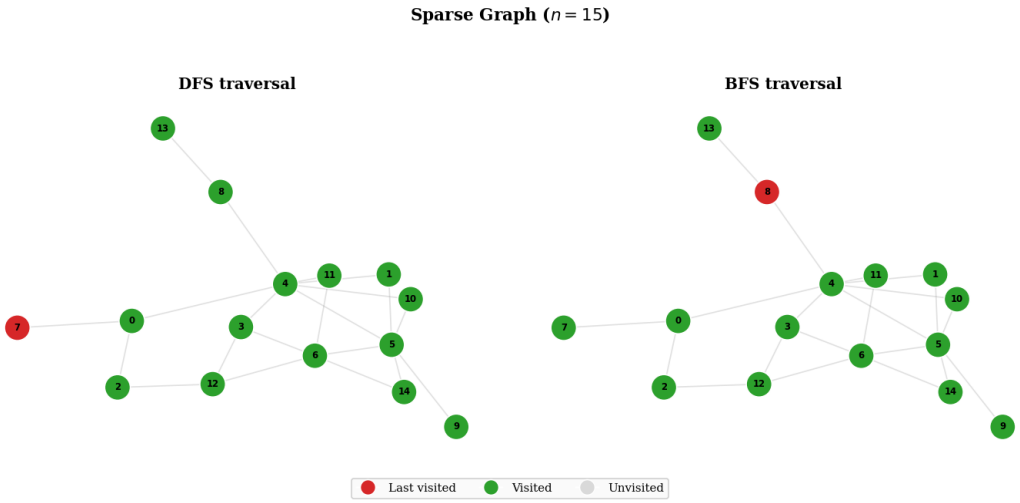

Figure 15: *Sparse Random Graph Traversal.*

Final DFS and BFS visitation states on a connected sparse random graph with $n = 15$, where both strategies have similar coverage cost.

12. Dense Random Graph

Erdős–Rényi $G(n, 0.7)$; $E \approx 0.35\, n^2$. Most nodes have degree close to $0.7\,(n-1)$.

DFS behaviour. With high connectivity, DFS quickly reaches every node; the stack peaks at $O(n)$. The dominant cost is iterating over the large adjacency lists.

BFS behaviour. BFS enqueues many neighbours at each step; the queue also peaks at $O(n)$. Performance is nearly identical to DFS, with the $O(n^2)$ edge set dominating the constant-factor differences.

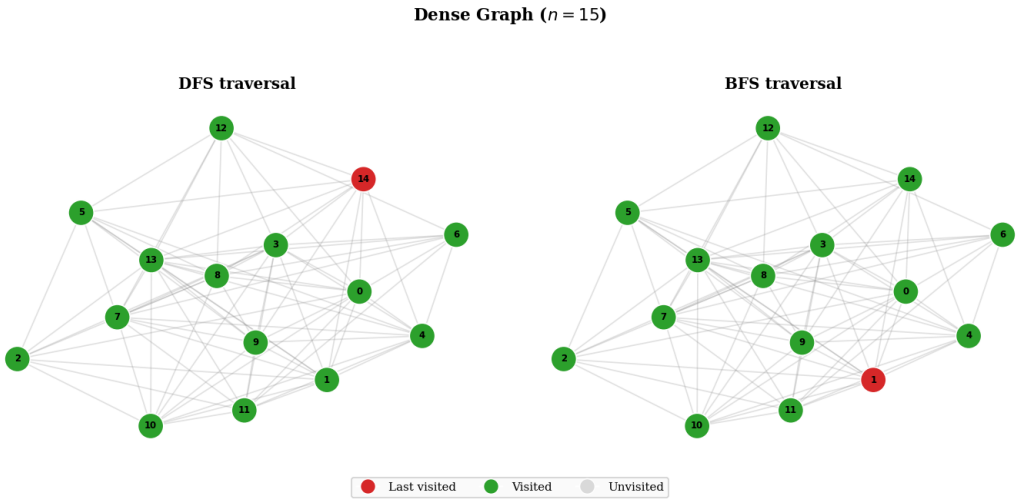

Figure 16: *Dense Random Graph Traversal.*

This figure shows that in a dense random graph, both traversals reach near-complete coverage very quickly because each visited vertex exposes many unvisited neighbors at once. In the DFS panel, the traversal still appears depth-oriented in ordering, but the high branching factor causes frequent local branching and rapid saturation of the visited set. In the BFS panel, the first few queue expansions already include a large fraction of the graph, so the frontier broadens aggressively and then collapses once few unseen vertices remain. The visual similarity of the final states confirms that the key difference is not the set of visited nodes but the order and intermediate frontier dynamics, while the main runtime cost comes from scanning large adjacency lists rather than from stack-versus-queue operations.

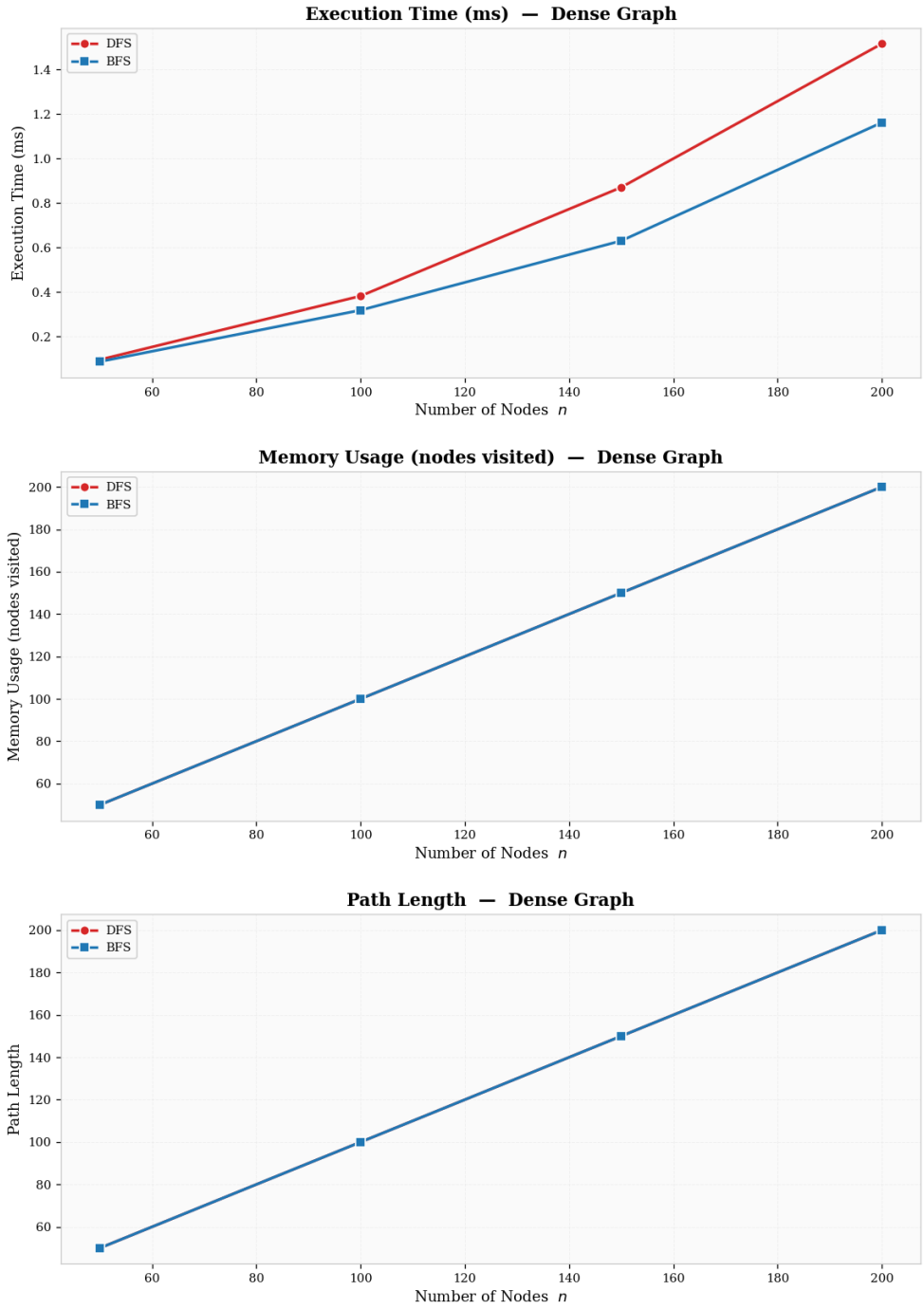

Figure 17: *Dense Random Graph Performance Comparison.*

DFS and BFS execution-time growth on dense random graphs, with super-linear increase driven by the $O(n^2)$ edge set.

13. Simple Graph

A simple graph is a connected undirected graph with no self-loops and no parallel edges – it represents the canonical definition of a graph. Here it is generated as an Erdős–Rényi

$G(n, 0.15)$ instance guaranteed to be connected.

DFS behaviour. With low average degree ($\approx 0.15\,(n-1)$), DFS tends to follow long paths before backtracking. The stack depth varies with the diameter of the particular random instance.

BFS behaviour. BFS expands outward through the sparse structure level by level. Both algorithms display nearly identical linear growth, mirroring the sparse random graph case but with a lower average degree.

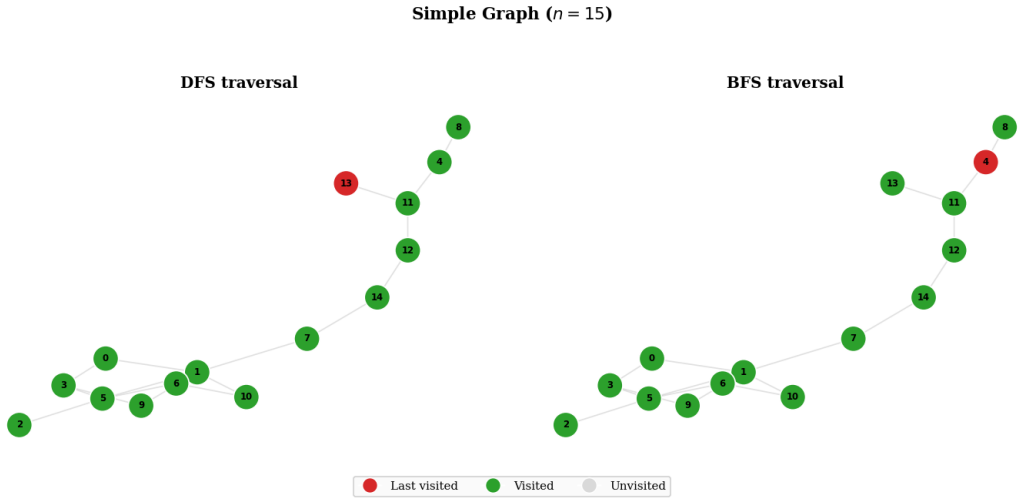

Figure 18: *Simple Graph Traversal.*

Final DFS and BFS visitation states on a simple connected sparse graph with $n = 15$, showing characteristic path-following (DFS) versus expanding-frontier (BFS) behaviour.

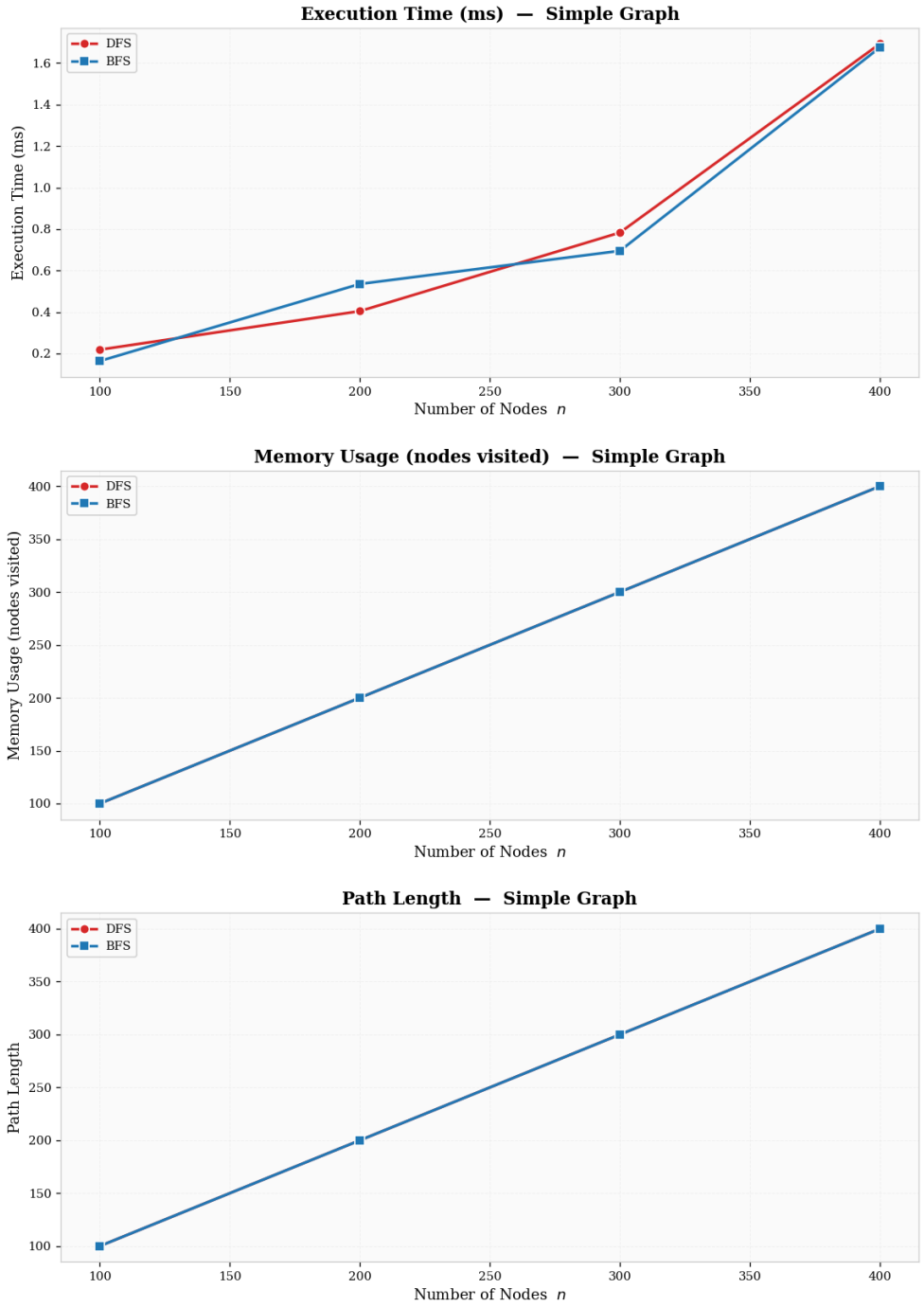

Figure 19: *Simple Graph Performance Comparison.*

Execution time trends on a simple sparse random graph, confirming linear scaling for both traversals with minimal constant-factor separation.

14. Multigraph

A multigraph allows multiple edges between the same pair of nodes. The instance here is built on a path backbone where every edge is duplicated, with additional skip edges

every third step. For traversal, parallel edges are collapsed to a simple graph (each node pair appears at most once in the adjacency list).

DFS behaviour. After collapsing multi-edges the effective graph is a path with sparse chords, so DFS behaves similarly to a path graph with slightly reduced stack depth due to skip edges.

BFS behaviour. Skip edges shorten the diameter, causing BFS to reach distant nodes in fewer levels. Both algorithms remain $O(V + E)$.

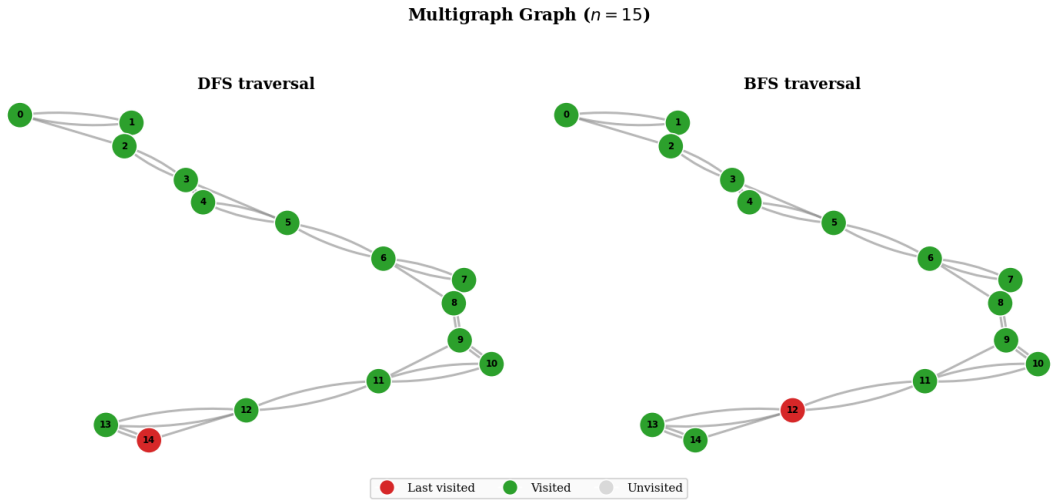

Figure 20: *Multigraph Traversal.*

Final DFS and BFS states on a multigraph (collapsed to simple) with $n = 15$, illustrating how shortcut chords alter traversal depth and frontier width.

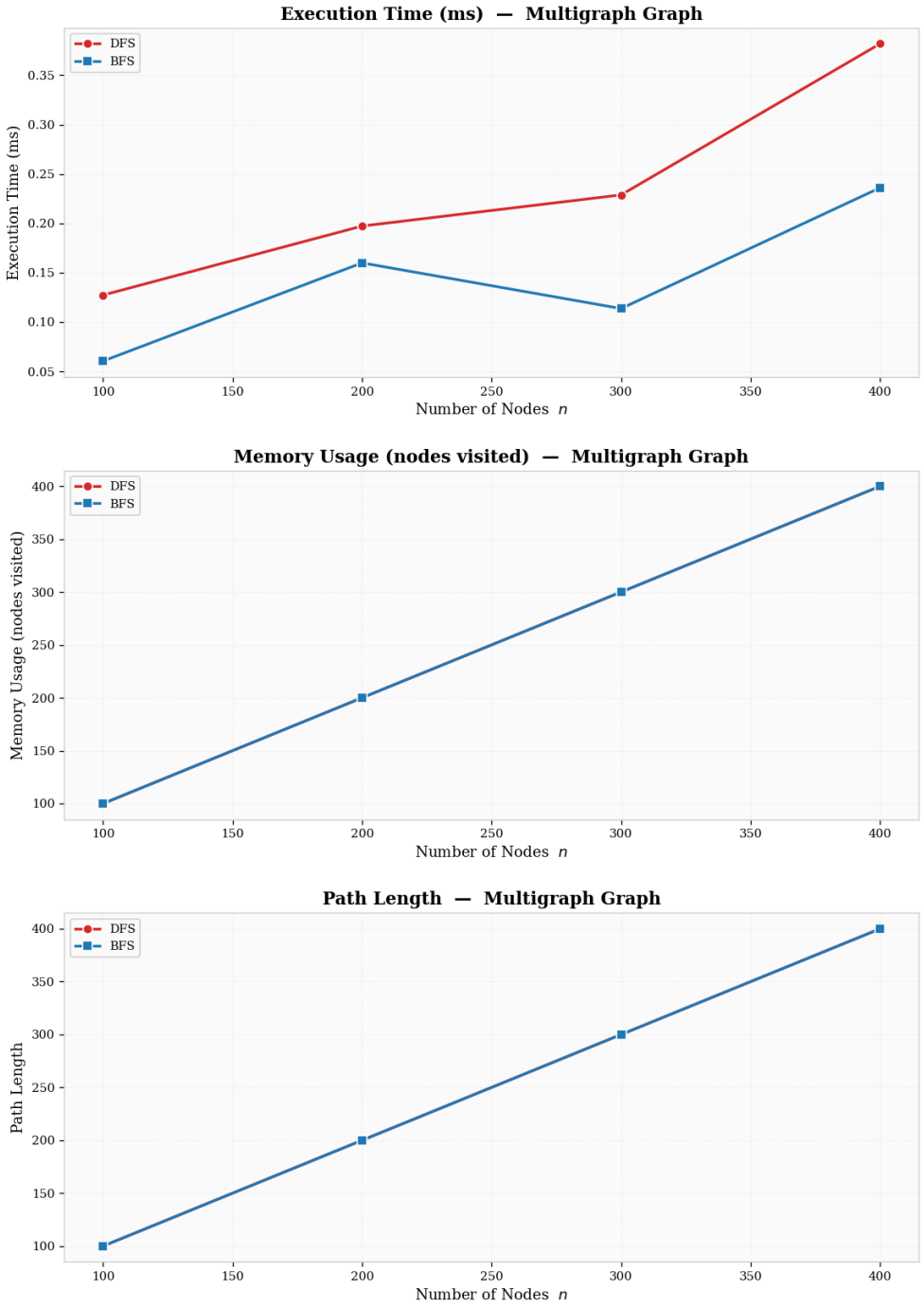

Figure 21: *Multigraph Performance Comparison.*

Execution time growth on the multigraph, showing how chord edges reduce both DFS stack depth and BFS diameter compared to a pure path.

15. Pseudograph

A pseudograph further extends the multigraph by permitting self-loops. Self-loops are removed before traversal (a self-loop adds no new reachable node); the resulting structure

retains parallel edges collapsed into a simple graph.

DFS behaviour. Comparable to the multigraph case. Self-loops add no new connectivity, so removal does not change the DFS order.

BFS behaviour. Equally unaffected by self-loop removal. The comparison isolates the effect of the multi-edge structure alone.

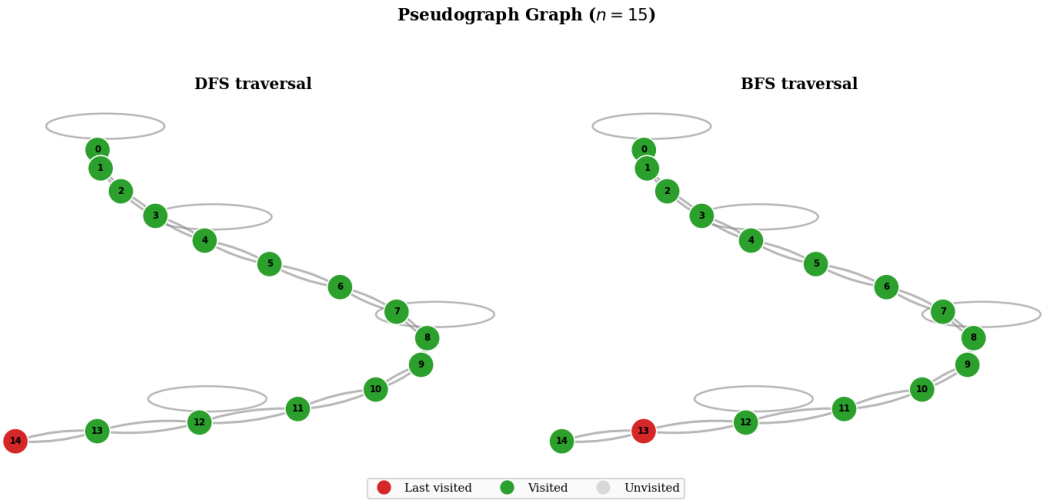

Figure 22: *Pseudograph Traversal.*

Final DFS and BFS visitation states on a pseudograph (self-loops removed) with $n = 15$.

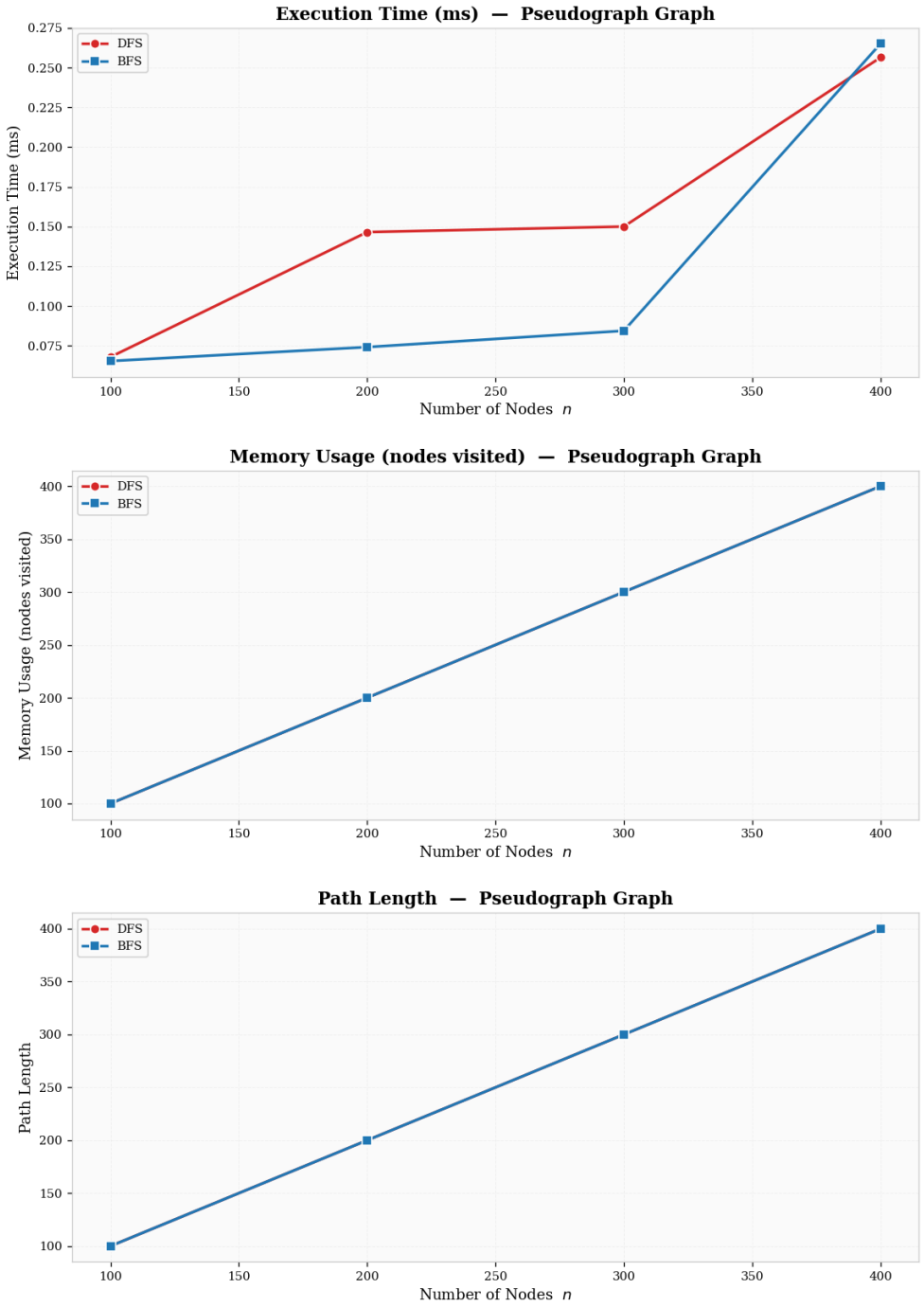

Figure 23: *Pseudograph Performance Comparison.*

Execution time on the pseudograph closely matches the multigraph, confirming that self-loops do not affect $O(V + E)$ traversal cost once removed.

16. Directed Multigraph

A directed multigraph has parallel arcs between the same ordered pair of nodes. The instance is built as a directed path backbone with every arc doubled, plus additional skip

arcs every third step. Parallel arcs are collapsed to a simple digraph for traversal.

DFS behaviour. The directed structure prevents backward traversal. DFS follows arcs forward, with skip arcs occasionally jumping two nodes ahead. Stack depth is bounded by the longest directed path.

BFS behaviour. BFS expands forward through levels; skip arcs reduce the number of levels. Performance is similar to the DAG case.

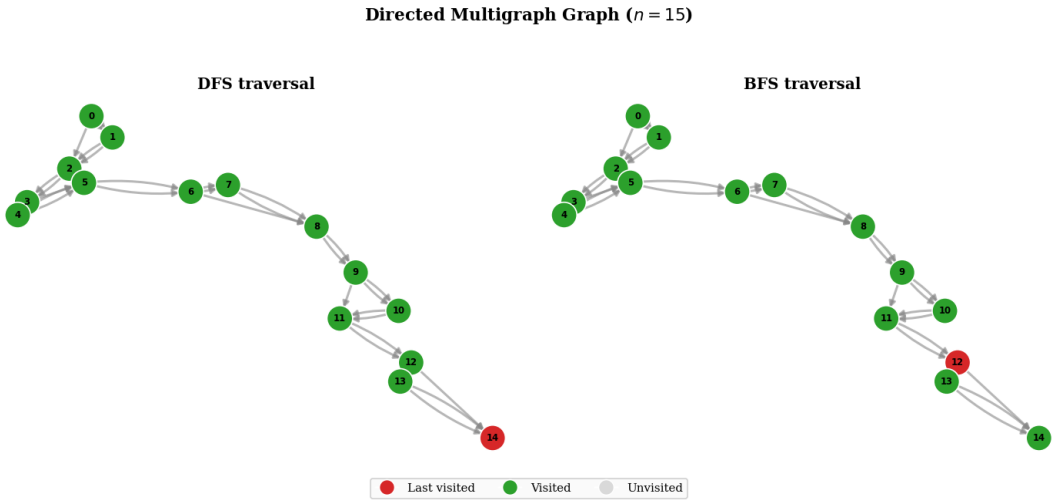

Figure 24: *Directed Multigraph Traversal.*

Final DFS and BFS states on a directed multigraph (collapsed) with $n = 15$, showing forward-directed exploration with occasional skips.

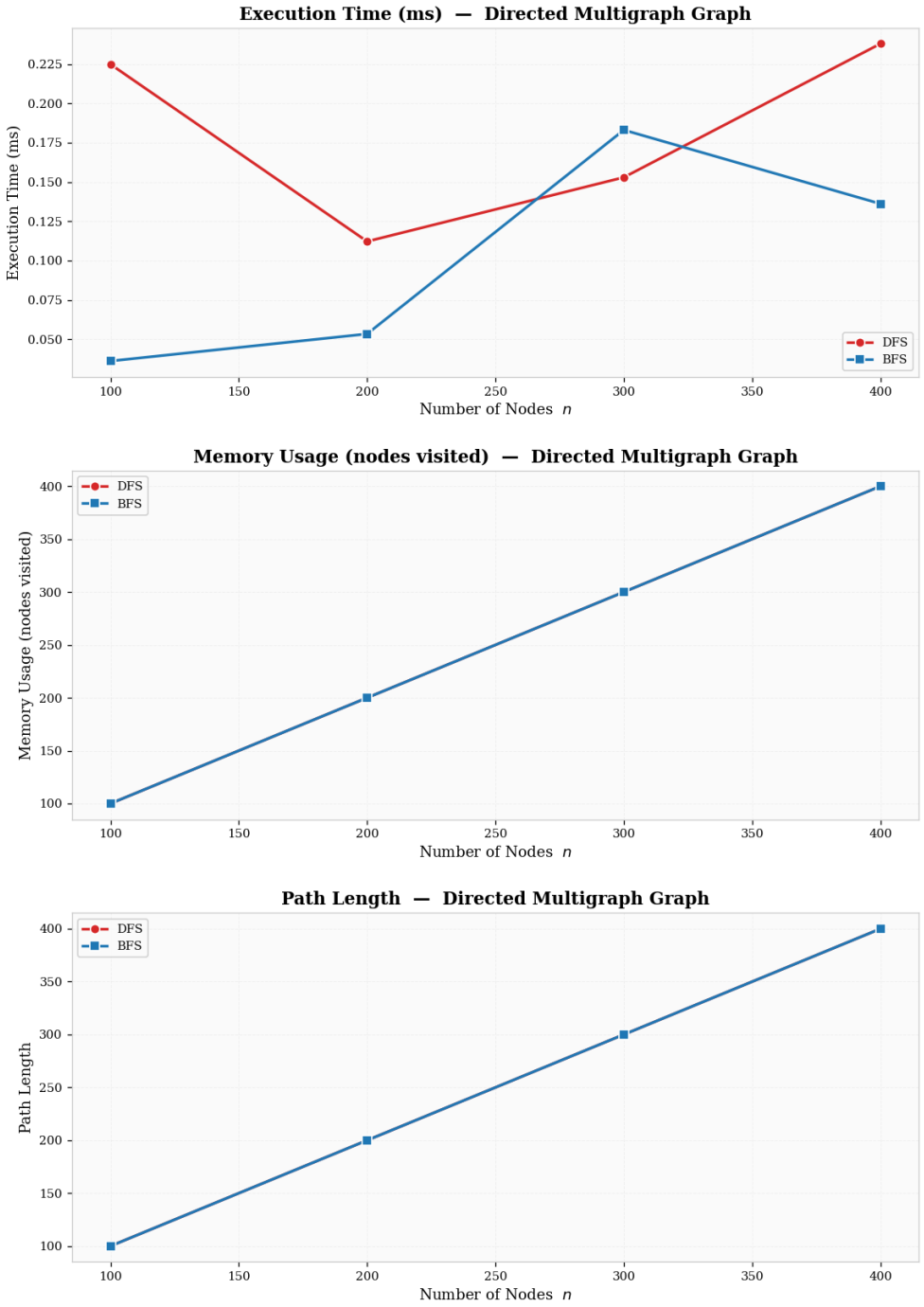

Figure 25: *Directed Multigraph Performance Comparison.*

Execution time trends on the directed multigraph, mirroring DAG behaviour with slightly reduced path lengths due to skip arcs.

17. Wheel Graph

A wheel graph W_n connects a single hub node to every node of an $(n-1)$-cycle; $E = 2(n-1)$. Every spoke and rim edge provides two paths between any two rim nodes (via

the hub or via the rim), making the diameter of W_n equal to 2 for the hub and at most 3 between rim nodes.

DFS behaviour. Starting from the hub, DFS pushes all $n-1$ rim nodes onto the stack, then traverses the rim chain from one end. The stack peaks at $O(n)$ but the adjacency-list scans are light.

BFS behaviour. BFS visits the hub (level 0) and all rim nodes (level 1) in a single step, making the queue peak at $n-1$ and completing traversal in two levels. BFS is measurably faster on wheels because of the extremely short diameter.

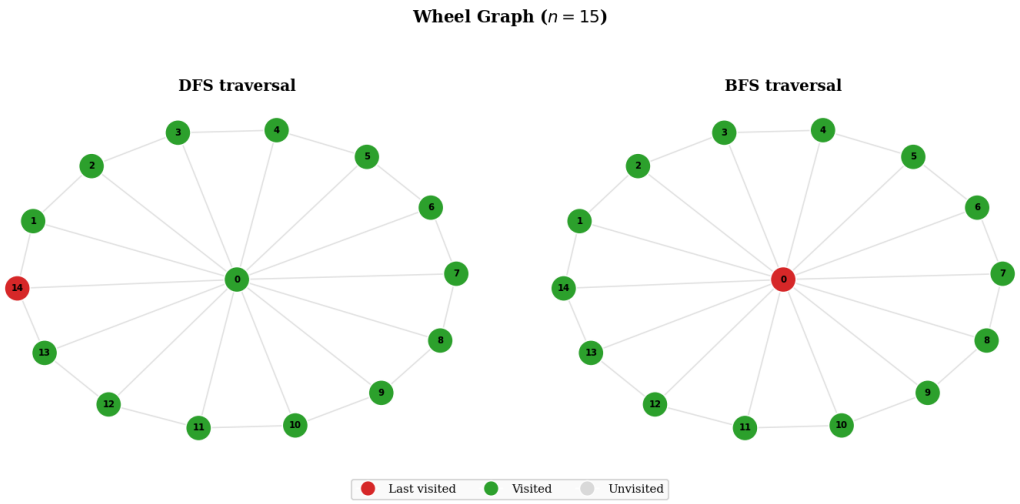

Figure 26: *Wheel Graph Traversal.*

Final DFS and BFS visitation states on a wheel graph with $n = 15$, emphasising hub-centric connectivity and two-level BFS completion.

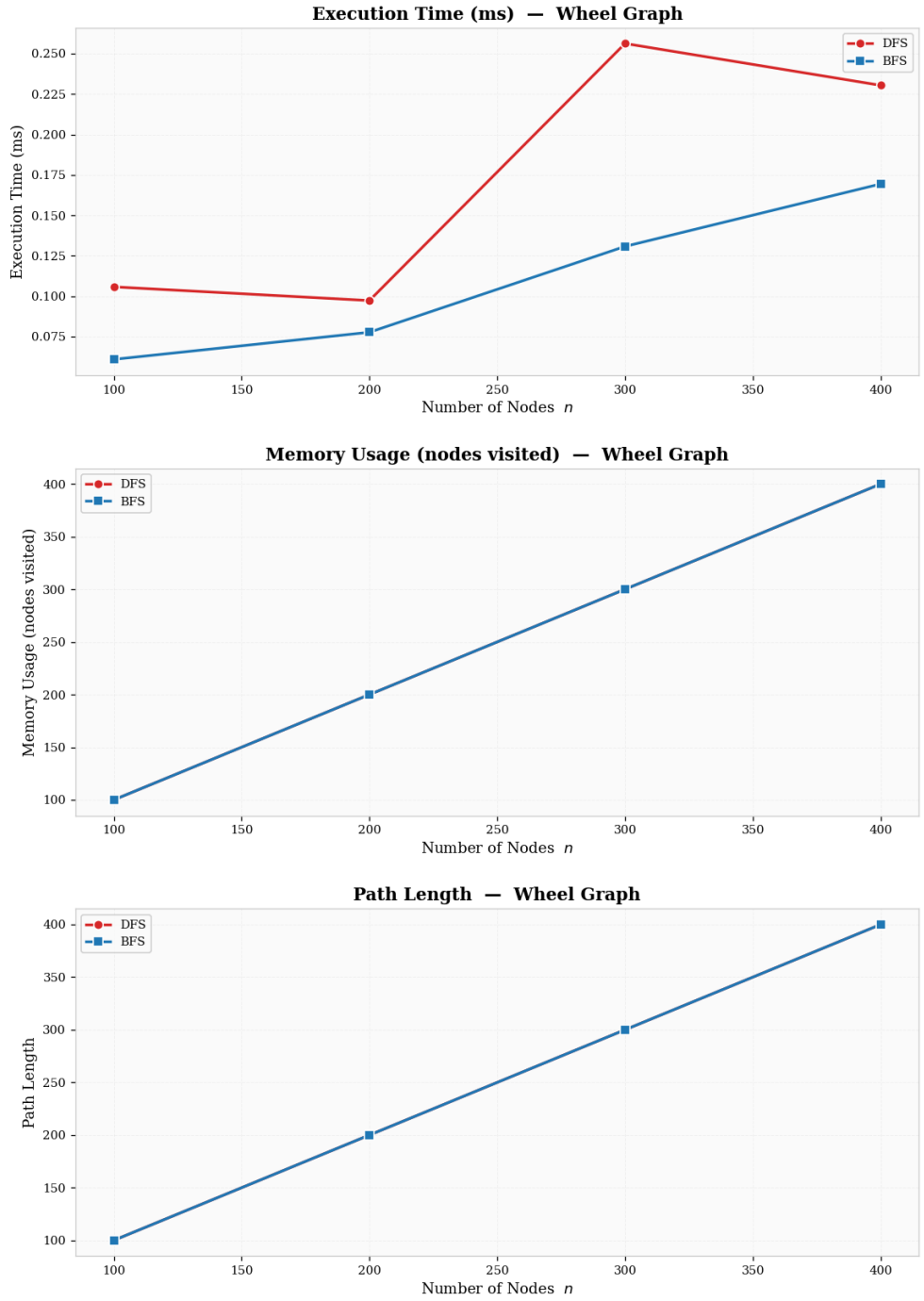

Figure 27: *Wheel Graph Performance Comparison.*

Execution time on wheel graphs, where BFS completes nearly instantaneously due to diameter 2, while DFS incurs linear stack overhead.

18. Regular Graph

A regular graph is one where every node has the same degree k. This instance uses $k = 3$ (a cubic graph) generated by `nx.random_regular_graph`; $E = 3n/2$. Uniform degree

eliminates topology-driven variance, making this graph ideal for isolating implementation-level differences between DFS and BFS.

DFS behaviour. With every node having exactly three neighbours, the stack pushes at most two new unvisited neighbours at each step. Stack depth scales with the longest DFS path in the random cubic structure.

BFS behaviour. Each BFS frontier expansion adds at most two new nodes per visited node. Both algorithms exhibit very similar constant factors, confirming that uniform degree neutralises topology-driven DFS/BFS differences.

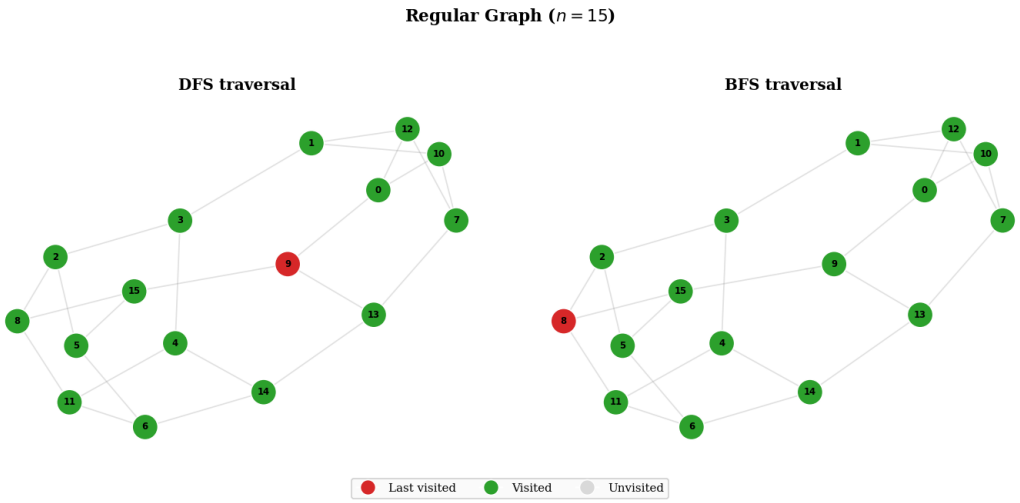

Figure 28: *Regular (Cubic) Graph Traversal.*

Final DFS and BFS states on a 3-regular graph with $n = 15$, where near-identical visitation patterns highlight uniform structural properties.

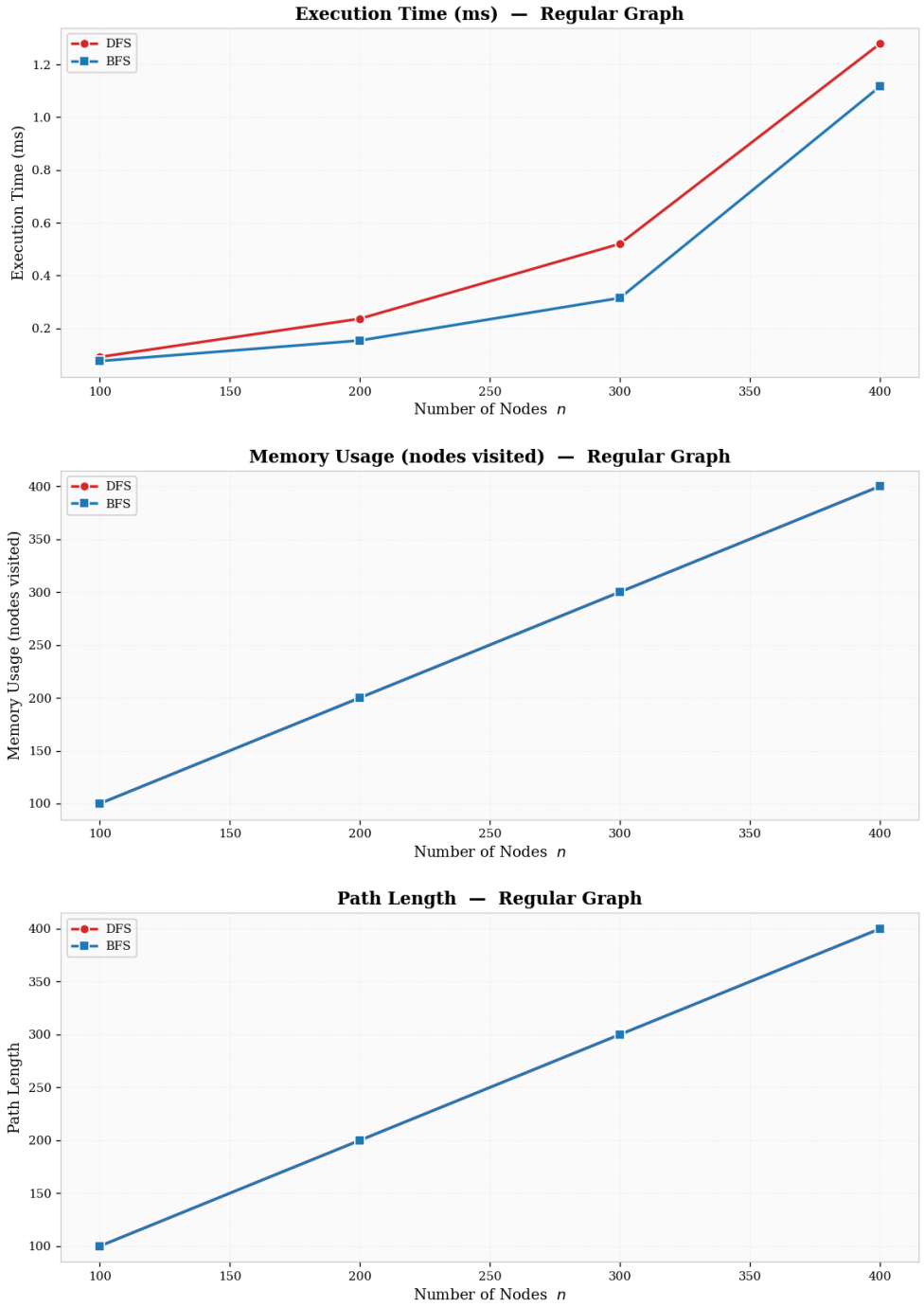

Figure 29: *Regular Graph Performance Comparison.*

Execution time on the cubic graph, showing minimal DFS/BFS separation – the clearest demonstration that equal degree neutralises traversal differences.

19. Weighted Graph

A weighted graph assigns a numeric cost to each edge. The instance here is a sparse connected random graph (same structure as the simple graph above) whose edges carry

integer weights drawn uniformly from [1, 20]. DFS and BFS *ignore* weights and explore the graph in unweighted traversal order, serving as a reference for what shortest-path algorithms such as Dijkstra would improve upon.

DFS behaviour. Explores by adjacency order, paying no attention to edge weight. Stack depth and total work are identical to the unweighted sparse random case.

BFS behaviour. Also ignores weights; finds a hop-minimal path but not a weight-minimal one. Both algorithms retain $O(V + E)$ complexity.

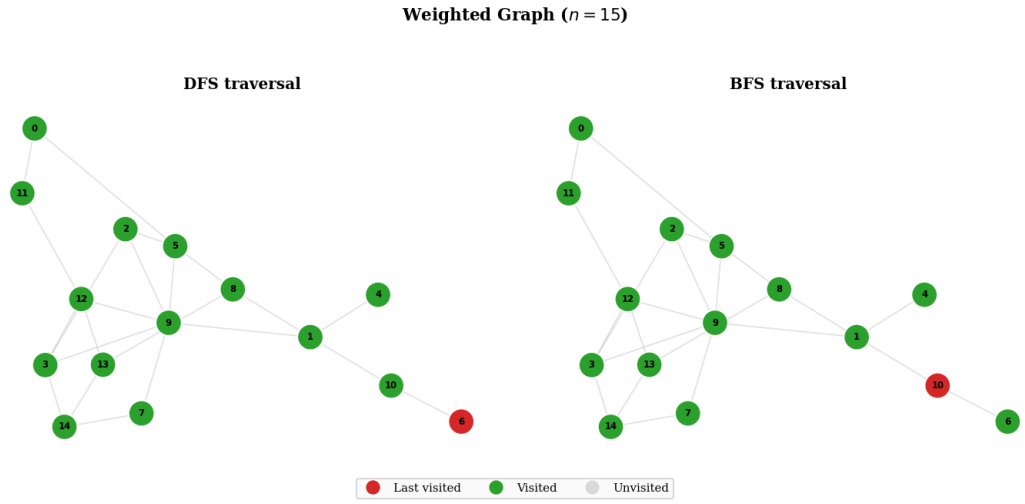

Figure 30: *Weighted Graph Traversal.*

Final DFS and BFS states on a weighted sparse random graph with $n = 15$. Edge weights are present but ignored during traversal.

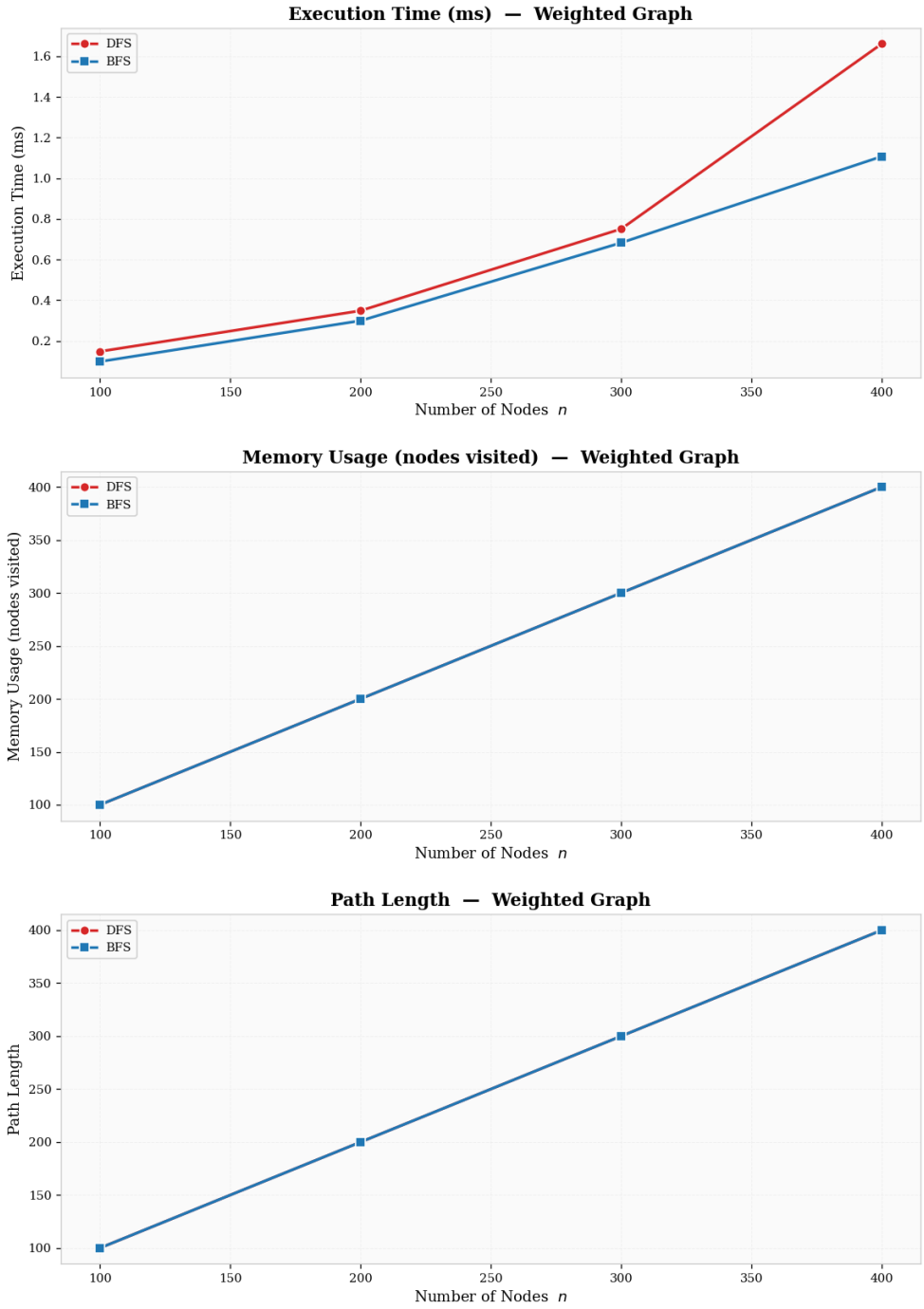

Figure 31: *Weighted Graph Performance Comparison.*

Execution time on the weighted graph mirrors the simple sparse random case, confirming that DFS and BFS runtime is unaffected by edge weight attributes.

Overall Summary

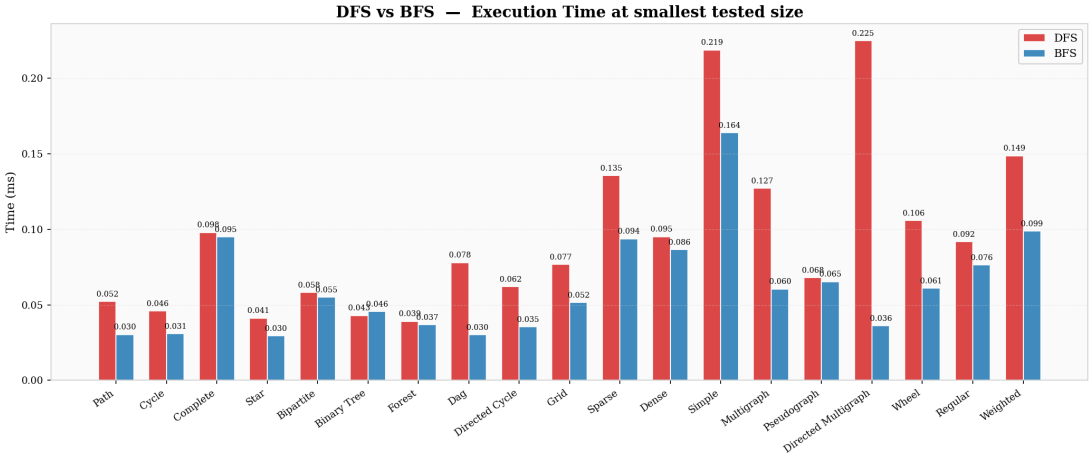

Figure 32: *Overall Time Comparison at Smallest Size.*

At the smallest tested size, this chart emphasizes constant-factor effects rather than asymptotic growth. DFS and BFS are close on most graph families, but slight separations appear where traversal structure strongly constrains frontier shape, such as path-like and tree-like cases. Dense families (complete, bipartite, dense random) already start at higher absolute times than sparse families because even small increases in node count imply many additional edge examinations. The figure therefore serves as a baseline: it captures the initial per-topology overhead profile before large-n scaling amplifies complexity differences.

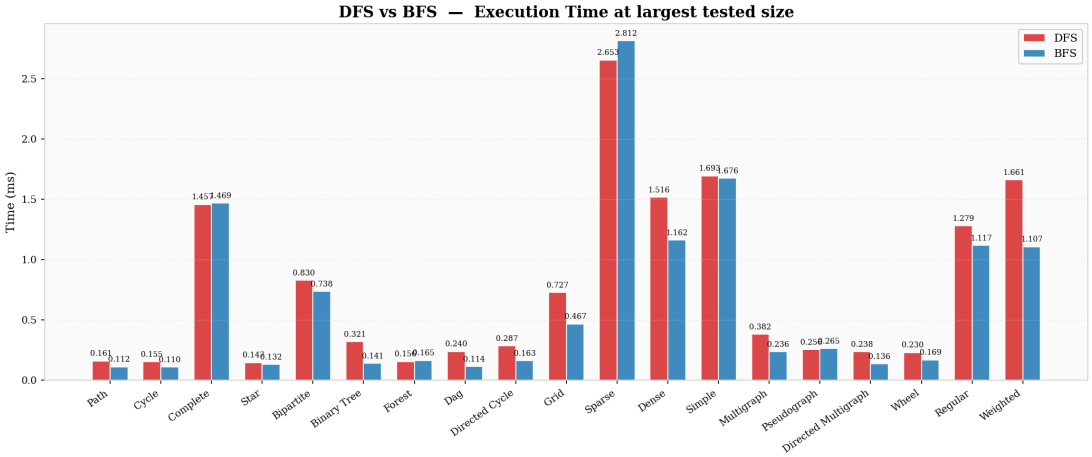

Figure 33: *Overall Time Comparison at Largest Size.*

At the largest tested size, the contrast between sparse and dense topologies becomes more pronounced. Graph classes with approximately linear edge growth remain comparatively low and preserve near-linear scaling, whereas dense classes rise sharply because edge

scanning dominates total work. The distance between DFS and BFS bars is generally still modest, indicating that both algorithms follow the same $O(V + E)$ law and differ mainly by implementation constants and exploration order. This figure highlights that topology choice has a larger impact on runtime than algorithm choice when both traversals are implemented efficiently.

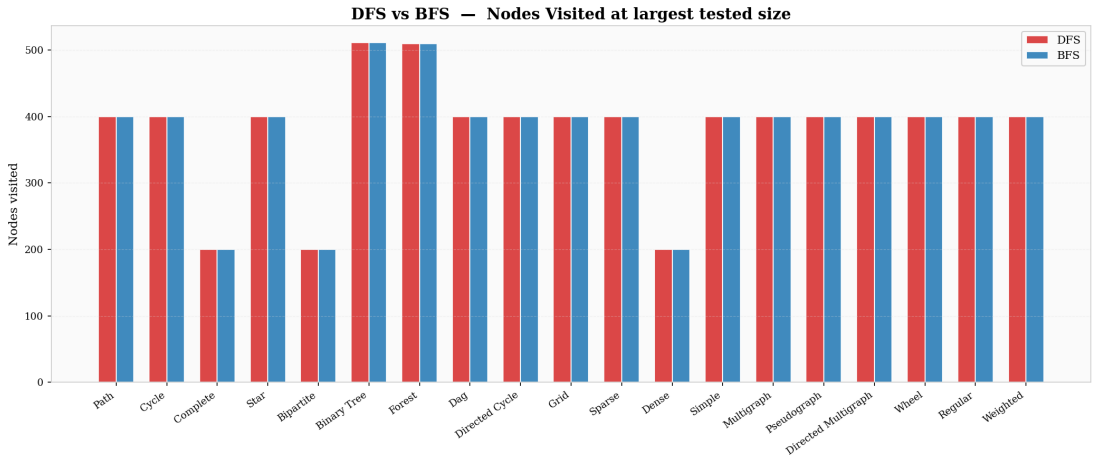

Figure 34: *Overall Visited-Node Comparison.*

This comparison confirms that, for connected inputs and full traversal runs, DFS and BFS visit essentially the same number of nodes, as expected from correctness of graph exploration. Any minor differences visible across categories are typically due to graph-generation details (for example, effective node counts after specific constructions) rather than a systematic memory advantage of one method. The figure should therefore be interpreted as a coverage-consistency check across all topology classes, while true memory-behavior differences are better explained by transient data-structure peaks, namely stack depth for DFS and frontier width for BFS.

CONCLUSION

Through this empirical study, both DFS and BFS were implemented and evaluated across nineteen representative graph categories, covering sparse, dense, structured, directed, multi-edge, weighted, and regular topologies.

Both algorithms share the same asymptotic complexity of $O(V + E)$, and the empirical measurements confirm linear growth in execution time as the number of nodes increases for all graph types where $E = O(V)$ (path, cycle, star, binary tree, forest, DAG, directed cycle, grid, simple, multigraph, pseudograph, directed multigraph, wheel, regular, weighted). On dense topologies ($E = O(V^2)$), complete, bipartite, dense random, both algorithms exhibit super-linear growth dominated by edge scanning.

The constant-factor differences observed in practice are driven mainly by the interaction between data-structure overhead and graph topology. Python's `collections.deque`, used by BFS, provides stable $O(1)$ front removals and good block-based locality, while the recursive DFS implementation relies on Python's call stack, where function-call overhead and recursion depth can become relevant on long path-like traversals. Topology then amplifies or reduces these effects: on path and cycle graphs DFS maintains a stack proportional to path length while BFS keeps a narrow queue; on star and wheel graphs BFS completes in only two levels while DFS must still visit all nodes sequentially; on balanced trees DFS retains only logarithmic-depth state while BFS can temporarily hold a large last-level frontier; on multigraphs and pseudographs the collapsed simple structure behaves like a sparse graph with chords; on regular graphs the uniform degree minimises DFS/BFS differences; and on dense graphs the dominant cost becomes edge scanning, which largely equalises the two algorithms.

From a practical perspective, DFS remains especially suitable for structure-oriented tasks such as cycle detection, topological reasoning, connected-component exploration, and backtracking-style searches where deep exploration is useful. BFS is preferable when shortest-hop paths are required, when level-order discovery is important, or when the target is expected to be close to the source in terms of graph distance. Weighted graphs require dedicated algorithms such as Dijkstra or Bellman-Ford for shortest-path queries; DFS and BFS provide only unweighted traversal on such structures. Therefore, the most effective strategy is topology-aware selection rather than rigid preference: BFS often provides better constants on path-like, wheel, and grid inputs, whereas DFS is often a strong choice for tree traversal and connectivity analysis.

In summary, for most general-purpose traversal tasks DFS and BFS perform comparably. The animated GIF visualisations generated for each graph type clearly show the contrasting exploration orders: DFS dives deep before backtracking, while BFS expands outward level by level.

REFERENCES

[1] **P., I. (2026).** *Analysis of Algorithms*. GitHub Repository. https://github.com/inarp235/AALab

[2] GeeksforGeeks. (2024). *Breadth First Search or BFS for a Graph*. https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

[3] GeeksforGeeks. (2024). *Depth First Search or DFS for a Graph*. https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/

[4] CS Dojo. (2018). *Introduction to Graph Theory: BFS and DFS* [Video]. https://www.youtube.com/watch?v=zaBhtODEL0w

[5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[6] Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.